Preliminary Design Tool for Flying Wing Unmanned Aerial Systems

Malik D. Henry

North Carolina Agricultural and Technical State University

A thesis submitted to the graduate faculty
in partial fulfillment of the requirements for the degree of

Master of Science
Program: Mechanical Engineering

Major Professor: John P. Kizito

Greensboro, North Carolina
2026

Abstract

This thesis presents the Flying Wing Tool, a software environment for the preliminary design of tailless aircraft built around the open-source AeroSandbox ecosystem and extended with custom geometry, structural, mission, and export capabilities. The motivation is simple: flying-wing design is highly coupled, but the preliminary workflow is often fragmented. Geometry is defined in one place, aerodynamics in another, mission assumptions in a third, and export in a fourth. The result is slow iteration, weak traceability, and repeated opportunities for the aircraft definition to drift.

The implemented system addresses this problem by organizing the design process around a shared project model. From that common representation, the software supports parametric geometry definition, spanwise section generation, twist-distribution synthesis, aerodynamic analysis, wingbox-oriented structural assessment, propulsion and mission evaluation, and export to downstream CAD and manufacturing formats. The central contribution of the thesis is therefore not a single new solver. Rather, it is a coherent preliminary-design workflow that keeps these capabilities connected.

The manuscript is intentionally disciplined in scope. It documents capabilities that are presently implemented and inspectable in the repository, distinguishes them from areas that still require validation, and uses a representative case study to show how the workflow behaves on a concrete aircraft definition. In doing so, the thesis positions the Flying Wing Tool within the broader literature on conceptual aircraft design, low-order aerodynamics, multidisciplinary workflows, structural sizing, mission analysis, and engineering-tool interoperability.

Contents

Contents ii

List of Figures vi

List of Tables vii

1 Overview and Problem Statement 1

1.1 Background and Motivation 1

1.2 Problem Definition and Research Gap 2

1.3 Research Question and Objectives 2

1.4 Scope of the Current Draft 4

1.5 Organization of the Thesis 4

2 Expanded Literature Review 5

2.1 Introduction 5

2.2 Summary of Reviewed Sources 5

2.3 Multi-Fidelity Preliminary Design and System-Level Coupling 9

2.3.1 Multi-fidelity MDO for conceptual design 9

2.3.2 Cross-tool comparison as verification 10

2.4 Aerodynamic Analysis for UAS and Flying Wings 11

2.4.1 What XFLR5 is designed to do (and what it is not) 11

2.4.2 Potential-flow solver comparison on a UAV configuration 11

2.4.3 VSPAERO validation and mesh recommendations 12

2.5 Flying-Wing Configuration, Stability, and Design Optimization 14

2.5.1 Optimization with explicit stability constraints 14

2.6 Differentiable Analysis and Optimization-Oriented Tooling 15
2.6.1 AeroSandbox and glass-box optimization requirements 15
2.7 Mission Modeling, Interoperability, and Export 15
2.8 Propulsion and Energy Storage Models 16
2.8.1 A battery model that is simple but parameterizable 16
2.9 Structural Sizing in Preliminary Design 17
2.10 Synthesis: Requirements for a Useful Flying-Wing Design Tool 17

3 Implemented Software Architecture and Physical Methods 19
3.1 System Architecture Overview 19
3.1.1 Main Layers 19
3.1.2 Primary Data Object 20
3.2 GUI Functionality 20
3.2.1 Main Window 20
3.2.2 Geometry Tab Group 21
3.2.3 Analysis Tab 21
3.2.4 Mission Tab 22
3.2.5 Structure Tab 22
3.2.6 Export Tab 22
3.3 Core Analysis and Modeling Services 24
3.3.1 Geometry and Aerodynamics Service 24
3.3.2 Airfoil Analysis Service 24
3.3.3 Aero Model Abstractions 25
3.3.4 Mission Subsystem 25
3.3.5 Propulsion Subsystem 25
3.3.6 Structural Subsystem 26
3.4 Export and Interoperability 27
3.4.1 Geometry Builder 27
3.4.2 Target Formats 27
3.5 Physical Methods Used 27

3.5.1 Variable Definitions (Nomenclature) 28
3.5.2 Aerodynamics 31
3.5.3 3-DOF Mission Dynamics 32
3.5.4 Ground Roll and Transition Physics 33
3.5.5 Propulsion Physics 34
3.5.6 Structural Mechanics 36
3.6 Data Inputs and Outputs 38
3.6.1 Inputs 38
3.6.2 Outputs 38

4 Integrated Workflow, Design Use Cases, and Contribution 40
4.1 Shared-State Workflow as the Core Contribution 40
4.2 Geometry-to-Analysis Traceability 40
4.3 Why Integration Matters More for Flying Wings 41
4.4 Representative Design Loops Enabled by the Current Codebase 42
4.5 Research Contribution of the Software Artifact 43

5 Representative Case Study: IntendedValidation2 44
5.1 Purpose of the Case Study 44
5.2 Case Definition 44
5.3 Aerodynamic Performance Results 46
5.4 Optimized Twist Distribution 46
5.5 Structural Results Under a 2.5g Load Case 48
5.6 Stored Mission Summary 49
5.7 Initial BARAM CFD Cross-Check 50
5.8 Interpretation of the Case Study 54

6 Validation Strategy, Limitations, and Discussion 55
6.1 What the Current Repository Demonstrates 55
6.2 What Still Requires Formal Validation 55
6.2.1 BARAM Mesh-Sensitivity Verification 56

6.2.2 Validated Interpretation of the Drag Discrepancy 58
6.2.3 Why a DATCOM-Like Buildup Method Is Weak for This Configuration 60
6.3 Accuracy Boundaries and Solver Assumptions 62
6.4 Relevance to Graduate Research and Design Practice 62
6.5 Discussion of Scope Discipline 63

7 Conclusion and Future Work **64**
7.1 Conclusion 64
7.2 Future Work 64

References **66**

Bibliography **67**

A Repository-Derived Scope of the Draft **70**

B Primary Repository Modules Referenced in This Draft **71**

List of Figures

1.1 Fragmented versus integrated preliminary-design workflow. 3

3.1 System architecture of the Flying Wing Tool. 20

3.2 Representative GUI views of the Flying Wing Tool. 23

3.3 Geometry and export views for interoperability workflows. 28

4.1 Shared-state workflow of the Flying Wing Tool. 41

5.1 Isometric geometry view of the case-study configuration. 45

5.2 Computed cruise and takeoff lift polars for `IntendedValidation2`. 47

5.3 Computed cruise and takeoff drag polars for `IntendedValidation2`. 47

5.4 Optimized twist distribution for `IntendedValidation2`. 48

5.5 BARAM and ParaView view of the CFD cross-check case. 51

5.6 BARAM monitor history re-scaled to the `IntendedValidation2` cruise reference quantities. 52

5.7 Comparison of re-scaled BARAM cruise estimates with the repository cruise point. 53

6.1 Planform-area share versus split-patch drag share for `IntendedValidation2`. 59

List of Tables

3.1 Primary variables and constants used in this report. 28

5.1 Selected geometry and trim parameters for the `IntendedValidation2` case. . 45

5.2 Computed aerodynamic operating-point results for `IntendedValidation2`. . 46

5.3 Selected structural results for `IntendedValidation2` at 2.5g. 49

5.4 Stored mission summary associated with `IntendedValidation2`. 50

5.5 Initial BARAM CFD cross-check against the repository cruise point for `IntendedValidation2`. 51

5.6 BARAM drag decomposition for the final 20 monitored iterations, converted to a full-aircraft estimate. 52

6.1 Matched BARAM mesh-sensitivity results for `IntendedValidation2`, with the former fine level relabeled as Medium 2 and the former extra-fine level reported as Fine. 57

6.2 Comparison of projected planform share and split-patch CFD drag share for `IntendedValidation2`. 58

Chapter 1

Overview and Problem Statement

1.1 Background and Motivation

Flying-wing aircraft continue to attract attention because they promise high aerodynamic efficiency, reduced wetted area, and favorable lift-to-drag performance. Those same advantages produce a difficult design problem. In a tailless configuration, the wing must provide not only lift but also trim authority, acceptable static stability, controllability, and a structurally practical load path. Planform choices therefore cannot be evaluated in isolation. Sweep, twist, airfoil selection, control-surface layout, and center-of-gravity targets all interact from the earliest stages of design.

This coupling creates a workflow problem as much as a physics problem. In many preliminary studies, the designer moves between low-order aerodynamic tools, hand calculations or spreadsheets, mission-performance scripts, and CAD or manufacturing software. Each handoff creates an opportunity for inconsistency. Geometry may be rebuilt manually, reference quantities may drift, and assumptions that were valid in one tool may not be carried faithfully into the next. These issues exist in many aircraft projects, but they are especially pronounced for flying wings because modest geometric changes can alter trim, loading, and manufacturability at the same time.

The present work was motivated by that gap. The point is not simply to add another solver to the process. The point is to provide a preliminary-design environment in which one aircraft definition can persist across geometry creation, aerodynamic analysis, structural reasoning, mission evaluation, and export. In other words, the value proposition is continuity of state across the preliminary design loop. Python is a practical foundation for such an envi-

ronment because it supports rapid integration across disciplines, and AeroSandbox provides a modern basis for geometry-aware analysis and optimization-driven workflows [4, 8].

1.2 Problem Definition and Research Gap

The core engineering problem can be stated simply: preliminary flying-wing design still lacks an open, extensible workflow that preserves consistency across geometry generation, aerodynamic analysis, structural sizing, mission evaluation, and downstream export. Existing tools are often effective within their own domains, but they usually capture only one slice of the design problem. Aerodynamic tools are fast but narrow, CAD environments are geometrically expressive but cumbersome for repeated trade studies, and mission or propulsion analyses frequently live apart from the geometry that defines the vehicle.

The resulting research gap is methodological as much as technical. Early in the design process, the engineer needs to ask coupled questions and answer them without rebuilding the aircraft each time. If the planform changes, what happens to trim twist and cruise performance? If the twist distribution changes, how do structural margins and manufacturing geometry respond? If propulsion assumptions shift, how does mission feasibility move relative to the same geometry and mass model already under study? A fragmented workflow can answer these questions, but only slowly and often with poor traceability.

The repository examined in this thesis addresses that gap by organizing the workflow around a shared project schema and a service-based analysis stack. The aim is not to replace every external tool. It is to establish a coherent core environment for flying-wing preliminary design, and then to export outward only when higher-fidelity analysis or downstream fabrication tasks require it.

1.3 Research Question and Objectives

The guiding research question for the thesis is:

> How can a unified software environment support parametric geometry definition, preliminary aerodynamic and structural analysis, mission-oriented evaluation,

Figure placeholder
`workflow_fragmented_vs_integrated.png`

Figure 1.1: Fragmented versus integrated preliminary-design workflow.

and downstream export for flying-wing aircraft without forcing the user to rebuild the aircraft model between stages?

The working thesis is that a shared parametric project model, coupled to AeroSandbox-based aerodynamic methods and custom structural, mission, and export services, can substantially reduce workflow fragmentation while remaining rich enough to support meaningful preliminary design iteration.

The main objectives of the work are as follows:

1. Develop a parametric aircraft representation that can support both conventional tailless wings and blended centerbody variants.

2. Connect that representation to aerodynamic analysis methods suitable for rapid preliminary trade studies.

3. Link aerodynamic outputs to structural assessment using a wingbox-oriented preliminary sizing framework.

4. Support propulsion-aware mission reasoning inside the same project environment.

5. Export geometry and manufacturing-oriented outputs directly from the shared aircraft representation.

1.4 Scope of the Current Draft

This draft is intentionally grounded in the repository as it exists at present. The text therefore distinguishes between three categories of content:

- implemented and central capabilities, which are treated as present contributions;
- implemented but still validation-limited capabilities, which are discussed carefully and with explicit boundaries;
- non-central or less mature extensions, which are deferred to future work rather than presented as completed thesis outcomes.

This distinction is important because the credibility of the manuscript depends on claim discipline. The repository already contains enough material to support a substantial thesis draft, but the argument is strongest when it remains aligned with what is currently implemented, inspectable, and reproducible in code.

1.5 Organization of the Thesis

The remainder of the manuscript proceeds as follows. Chapter 2 incorporates and extends the existing literature review assembled for the project, with emphasis on methods and tools relevant to integrated flying-wing preliminary design. Chapter 3 documents the implemented software architecture and the physical methods embedded in the current repository. Chapter 4 argues for the software contribution in workflow terms and explains why integration is itself the primary result. Chapter 5 presents a representative case study based on the `IntendedValidation2.json` project. Chapter 6 discusses validation strategy, limitations, and the boundaries of the current claims. Chapter 7 closes with the principal conclusions and the most relevant next steps. The appendices summarize repository modules and the scope assumptions used while drafting.

Chapter 2

Expanded Literature Review

This chapter incorporates and adapts the repository's prior literature survey. The goal is not merely to accumulate sources, but to position the software contribution within existing work on conceptual aircraft design, low-order aerodynamic tools, multidisciplinary workflows, mission modeling, structural sizing, and interoperability.

2.1 Introduction

Flying-wing UAS design forces you to confront coupling early. Planform and twist choices affect trim and static margin. Stability choices affect control surface deflections and spanwise loading, which changes structural margins and the aerodynamic drag picture. Propulsion and energy storage choices change mass properties and mission feasibility, and those changes feed back into the aero and structure.

This literature review focuses on methods and tools that directly support an integrated preliminary design workflow for flying-wing aircraft. The emphasis is on what the sources actually do, what they assume, and where they tend to fail. The goal is to extract guidance for a tool that connects geometry definition, aerodynamic analysis, mission simulation, structural checks, and export to downstream environments.

2.2 Summary of Reviewed Sources

Source	What it does	Key details used in this review	Access notes
Nguyen et al. (2013) [1]	Multi-fidelity MDO for a UCAV concept using a CFD-backed metamodel inside an MDF loop.	D-optimal DOE for CFD points; CFD cruise condition reported as Mach 0.85 at 38,000 ft and 2 deg; DOE size reported as 20 configs; metamodel used to avoid CFD-in-the-loop.	Often paywalled (journal); look for author copy or institutional access.
Vegh et al. (2019) [12]	Cross-tool comparison (NDARC vs SUAVE) on Kitty Hawk Cora with sweeps and SLSQP optimization.	Baseline GTOW reported (2233 lb vs 2086 lb); wingspan sweep trend disagreement attributed to drag; optimized designs and cross-evaluation reported.	Public PDF available via SUAVE/Stanford.
Deperrois (2009) [13]	XFLR5 guidelines describing scope, assumptions, and numerical setup expectations.	Explicit warnings about applicability; emphasizes mesh/panel refinement and convergence checks.	Public PDF (XFLR5 documentation).

Continued on next page

Source	What it does	Key details used in this review	Access notes
Yu et al. (2022) [14]	Potential-flow solver comparison on T-FLEX UAV against STAR-CCM+ Euler/RANS baselines.	RANS uses Spalart-Allmaras; CFD mesh study 0.9–22M cells (12.3M chosen); potential-flow convergence criterion (1% CL change); notes on C_m and induced drag extraction differences.	Public PDF (ICAS archive).
Rosas-Cordova et al. (2024) [15]	VSPAERO validation and tessellation sensitivity on basic wings.	Spanwise tessellation drives convergence; recommends spanwise tessellation > 4 for unswept; reports lift-curve-slope errors (20.55% at AR=2, 6.19% at AR=4); provides a recommended tessellation parameter set and stated validity bounds.	Public PDF available via the publisher site mirror.
Zhang et al. (2025) [10]	Flying-wing glider optimization using ANN surrogate and GA with stability constraints.	Static margin constraint reported as 5%; reports performance penalty (11.5% range reduction) and example stabilized performance numbers.	MDPI article is typically open-access.

Continued on next page

Source	What it does	Key details used in this review	Access notes
Sharpe (2021) [4]	AeroSandbox thesis: differentiable analysis and optimization-first workflows for aircraft design.	Discussion of "glass-box" requirements under IPOPT; framing of SAND-style formulations; emphasis on robust analysis functions for optimization.	MIT thesis repository (sometimes blocked by network policies); may need manual download.
Tremblay et al. (2007) [20]	Generic Thevenin-style battery model for dynamic simulation with parameter identification from discharge curves.	Terminal-voltage model structure (OCV + polarization + exponential zone + ohmic drop); stated omissions (no hysteresis, no temperature dependence).	IEEE paper (often paywalled).
Berndt (2009) [39]	JSBSim flight dynamics model description and intended modeling scope.	Used to justify export-to-simulator as a verification path and to frame what is reasonable to expect from the external model.	AIAA paper (often paywalled).
Bruhn; Niu; NASA SP-8007; Timoshenko and Gere [30, 33, 29, 24]	Standard references for aircraft structural sizing and buckling checks.	Used to motivate the checklist of early structural outputs: stress, deflection, torsion, panel buckling, column/crippling margins.	Books (library access) and NASA report (public).

2.3 Multi-Fidelity Preliminary Design and System-Level Coupling

2.3.1 Multi-fidelity MDO for conceptual design

Nguyen et al. describe a multidisciplinary conceptual design approach for an unmanned combat air vehicle that explicitly mixes low-fidelity discipline models with a higher-fidelity aerodynamic correction strategy [1]. Their baseline conceptual loop couples aerodynamic estimation, weight estimation, and mission range inside a multidisciplinary feasible (MDF) iteration. Instead of embedding CFD inside the optimizer, they generate a limited set of higher-fidelity aerodynamic evaluations and fit a metamodel that the MDO process can query quickly.

Two technical choices in Nguyen et al. map well to a tool workflow. First, they do not sample high-fidelity points randomly. They use D-optimal design of experiments to choose configurations that improve metamodel quality per CFD run [1]. Second, they evaluate those CFD points at an explicitly stated cruise condition (reported as Mach 0.85, 38,000 ft, and 2 deg angle of attack) using Fluent, then use the metamodel in the optimizer [1]. The high-fidelity effort is used to correct a low-order model in a narrow but important operating regime.

They report improved aerodynamic prediction accuracy without a commensurate increase in design time because the optimizer never directly pays CFD cost [1]. The limitation is also structural: as the design moves away from the DOE region, the metamodel becomes an extrapolator. A robust workflow therefore needs both diagnostics (to detect leaving the training envelope) and a mechanism to add new high-fidelity points when needed.

In their implementation, the DOE is small by design (reported as 20 configurations for the high-fidelity database) and the optimization is posed as a multi-objective trade between mission range and aircraft weight [1]. The method is useful because it combines fast iteration with a higher-fidelity anchor while making the validity region explicit. The tool is only as trustworthy as the DOE coverage.

2.3.2 Cross-tool comparison as verification

Vegh et al. compare NDARC and SUAVE by modeling the Kitty Hawk Cora aircraft under a simplified mission definition [12]. The aircraft class differs from a flying wing, but their main contribution here is methodological: it shows how two established conceptual design tools can disagree in ways that change design decisions.

They report baseline gross takeoff weight (GTOW) predictions in the same range (2233 lb for NDARC and 2086 lb for SUAVE) while noting that the internal modeling choices differ in meaningful ways [12]. For example, SUAVE performs a mission simulation that converges an energy network at each time step and represents the battery using a Thevenin-equivalent circuit, whereas NDARC uses a more sizing-oriented approach [12]. When they sweep rotor radius, both tools show similar weight sensitivity. When they sweep wingspan, the tools show opposite trends, which they attribute primarily to differences in wing drag predictions [12].

The paper is also careful about defining the scenario it is comparing against. They collect public data for the Cora concept (including an 11 m wingspan) and use a simplified Uber Elevate mission definition with a representative payload (reported as 400 lb), a range target (100 km), and a cruise speed target (180 km/h), along with an operating altitude band (500 to 3000 ft) [12]. Those details matter because many disagreements between conceptual tools appear to be physics disagreements but are actually mismatches in mission definition and accounting conventions.

The paper also reports an optimization study using SLSQP. The resulting optima differ not only in the chosen design variables, but also in GTOW. They then cross-evaluate each optimized design in the other tool and show that the discrepancy persists [12]. The takeaway for an integrated preliminary design tool is that cross-checking is not a luxury. A tool should make it easy to reproduce intermediate quantities (lift, drag breakdowns, power flows, margins) so disagreements can be traced to assumptions, conventions, or numerical setup rather than being waved away.

One detail that makes the comparison actionable is that they report the optimized designs, not just the fact that they differ. In their study, the NDARC optimum is reported

at 2223 lb with a rotor radius of 1.83 ft and wingspan of 39.5 ft, while the SUAVE optimum is reported at 1948 lb with a rotor radius of 1.70 ft and wingspan of 30.0 ft [12]. Even if the exact numbers are not transferable to a flying wing, the pattern is: two tools can produce different optima under the same requirements because they disagree on intermediate modeling components like drag and energy usage.

2.4 Aerodynamic Analysis for UAS and Flying Wings

2.4.1 What XFLR5 is designed to do (and what it is not)

Deperrois' XFLR5 guidelines are direct about scope and limitations [13]. The document positions XFLR5 as a tool for model sailplanes and model aircraft and explicitly discourages using it for full-size aircraft design [13]. The software couples 2D airfoil analysis (via XFOIL-style methods) with 3D wing analysis using LLT, VLM, and a panel method. Viscous effects enter primarily through section polars rather than a full 3D viscous solution.

The guidelines emphasize that numerical choices matter: panel discretization and refinement influence results, and users are expected to run convergence checks [13]. For a flying-wing workflow, XFLR5 is useful as a fast trend tool when its assumptions hold. When designs approach stall, strong separation, or complex control-surface behavior, its outputs are best treated as hypotheses to test rather than answers to trust.

2.4.2 Potential-flow solver comparison on a UAV configuration

Yu et al. compare a set of potential-flow solvers on the T-FLEX UAV and evaluate the results against higher-fidelity CFD performed in STAR-CCM+ [14]. They include both Euler and RANS simulations, and for the RANS baseline they report using the Spalart-Allmaras turbulence model. They document a mesh study from 0.9 to 22 million cells and select a 12.3 million cell mesh as a convergence compromise [14].

The set of potential-flow tools they evaluate includes AVL, Tornado, XFLR5, VSPAERO, PAWAT, and FlightStream [14]. What the paper adds is a clear comparison of how those tools differ on the same configuration, rather than treating them as interchangeable.

A strength of the study is that they treat numerical convergence as part of the experiment. On the potential-flow side, they increase panel resolution until the change in lift coefficient between refinements is below 1% [14]. This matters because solver-to-solver differences can be smaller than the error induced by an under-resolved discretization.

Their findings are nuanced. In the linear regime, lift curve trends are generally consistent across tools, but drag and moment behavior is more sensitive to model structure and post-processing. They discuss that some tools incorporate boundary-layer corrections (FlightStream is described as using an integral boundary layer method), while others remain closer to inviscid potential flow [14]. For pitching moment results, they report that VSPAERO and FlightStream agree more closely with the CFD pitching moment slope for their configuration [14]. They also note that induced drag is not a single, universally extracted quantity in these codes; how induced drag is computed and reported can drive apparent disagreement even when lift predictions match [14].

For flying-wing design, this distinction is not academic. A design can look excellent in lift and induced drag and still be unusable because the pitching moment curve forces large trim deflections or produces an unstable neutral point. Yu et al.'s comparison implicitly ranks outputs: for conceptual trade studies, a solver that provides stable and repeatable $C_m($
$alpha)$ behavior is often more valuable than one that only provides C_L trends [14].

For a flying-wing preliminary design tool, these details matter because the design is often constrained by moment and stability behavior at least as much as by L/D. A solver backend that provides lift but gives unreliable moment trends can mislead an early design loop.

2.4.3 VSPAERO validation and mesh recommendations

Rosas-Cordova et al. validate VSPAERO on a set of basic wing configurations and focus on how tessellation choices affect results [15]. They provide mesh parameter recommendations and quantify where the method struggles.

They report that chordwise tessellation had little effect on their results in the studied cases compared with other uncertainties, while spanwise tessellation strongly affected

convergence. For unswept wings they recommend spanwise tessellation values above 4 for consistent convergence, and they note that swept wings require finer spanwise resolution [15]. They also summarize bounds where they consider results acceptable (for example, moderate angles of attack and higher aspect ratio cases) and explicitly call out regimes where errors grow [15].

They provide error magnitudes that are relevant to tool users. In a low Reynolds number case with aspect ratio 2, they report a 20.55% error in lift curve slope prediction, while a higher aspect ratio case (AR=4) shows a smaller error (6.19%) [15]. Flying wings often sit in the moderate-to-low aspect ratio regime, so this result should shape expectations. VSPAERO can be a fast baseline for planform trade studies, but results should be cross-checked for low-AR wings and for swept cases where tip stall and 3D separation effects become important.

They also compare VSPAERO outputs to experimental references (including classic NACA data for basic wing cases) and summarize where the method tends to under- or over-predict trends [15]. They make the point that validation is rarely one number. A tool can match lift curve slope while missing pitching moment slope, or match moment while missing drag. Their paper is most useful where it separates numerical setup effects (tessellation choices) from modeling-domain effects (aspect ratio, Reynolds number, sweep) [15].

They also provide a concrete starting point for meshing parameters that can be embedded in a tool as defaults. One recommended set includes chordwise tessellation of 33, spanwise tessellation of 24, chordwise cluster of 0.3, spanwise cluster of 0.6, wake length of 5 chord lengths, and wake tessellation of 24 [15]. They describe an intended validity regime that includes moderate angles of attack (reported as -10 deg to 10 deg), low Mach number ($M < 0.3$), Reynolds number above 100,000, and aspect ratio above 4 [15]. Those bounds are useful because they separate two questions a design tool user might otherwise conflate: whether the solver ran and whether the solver is expected to be accurate.

2.5 Flying-Wing Configuration, Stability, and Design Optimization

2.5.1 Optimization with explicit stability constraints

Zhang et al. present an optimization workflow for flying-wing gliders that treats stability as a constraint rather than a post-check [10]. Their approach uses an ANN-based surrogate model coupled with a genetic algorithm. They include a static margin requirement (reported as 5%) and evaluate the performance cost of enforcing it [10].

Methodologically, the paper is an example of "surrogate-first" preliminary design. The ANN stands in for repeated aerodynamic evaluations inside the genetic algorithm loop, which allows broader exploration of the design space than would be practical if each candidate required a slow solver call [10]. The tradeoff is that the optimizer inherits the surrogate's blind spots; stability and trim constraints therefore do double duty as both design requirements and as guardrails that reduce the chance of selecting an obviously unrealistic configuration.

Their results make two points that align with practical flying-wing design experience. First, enforcing stability has a measurable performance penalty. They report an 11.5% reduction in maximum range relative to an unstable optimum, while still achieving a reported glide ratio of 24.1 and a static margin of 5.1% [10]. Second, stability enforcement can drive geometry changes that are not obvious from aerodynamic efficiency alone, because the available degrees of freedom are limited when there is no tail volume.

They also illustrate a limitation common to surrogate-driven optimization: the surrogate is only as good as the training data and the aerodynamic model used to generate it. If the training method does not capture separation onset, control-surface effectiveness near stall, or post-stall moment behavior, the optimizer can select designs that look attractive in the surrogate but fail in higher-fidelity analysis.

2.6 Differentiable Analysis and Optimization-Oriented Tooling

2.6.1 AeroSandbox and glass-box optimization requirements

Sharpe's AeroSandbox thesis is explicit about building aircraft analysis around optimization rather than adding optimization later [4]. The implementation is built around algorithmic differentiation and nonlinear programming, using CasADi for differentiation and IPOPT as a primary solver [4].

A central practical issue discussed in the thesis is that IPOPT expects analysis functions to behave like "glass-box" mappings. Analysis functions must return outputs and gradients in a form that remains defined even when intermediate values wander into infeasible regions. This motivates robust extrapolation behavior, careful variable bounds, and constraint formulations that do not produce undefined expressions [4].

The thesis also motivates simultaneous analysis and design (SAND) formulations: instead of nesting analysis inside optimization, analysis state variables can be included as decision variables with constraints enforcing the governing equations [4]. In preliminary design contexts, SAND can reduce outer-loop iteration counts and make gradients more informative, but it increases the size of the nonlinear program and places more burden on numerical conditioning.

For a flying-wing workflow, the mapping is direct. Twist optimization, lift distribution shaping, and mission parameter tuning can be posed as constrained optimizations, but only if the underlying analysis functions stay well-behaved under optimizer pressure. Sharpe is explicit about where the effort goes if you want the optimization results to be trustworthy: not the optimizer call, but the analysis robustness [4].

2.7 Mission Modeling, Interoperability, and Export

Mission modeling often fails first at the interfaces. Vegh et al. show this indirectly: even when two tools target the same vehicle and similar mission requirements, differences in

aerodynamic drag modeling and mission simulation details can flip trade study trends [12]. For an integrated design tool, this suggests that mission simulation outputs should preserve intermediate quantities (forces, power, state-of-charge trajectories, margins) so the user can see why the answer changes.

If the workflow exports models to an external simulator, the internal modeling conventions must map cleanly to the simulator's configuration language. Berndt describes JSBSim as an open-source flight dynamics model and clarifies its intended modeling scope [39]. That makes it a reasonable target for early verification of trim and stability behavior. The export path then becomes part of the modeling contract: coordinate frames, sign conventions, reference areas, and control surface definitions must be explicit, consistent, and testable.

2.8 Propulsion and Energy Storage Models

2.8.1 A battery model that is simple but parameterizable

Tremblay et al. propose a generic battery model intended for dynamic simulation, built around a small set of parameters that can be identified from discharge curves [20]. The model represents the terminal voltage as a combination of an open-circuit voltage term, a polarization term that depends on extracted capacity, an exponential term to capture initial voltage drop behavior, and a series resistance drop [20]. The state of charge is tracked through current integration, and the formulation is designed to be computationally light enough for system simulation.

In their notation, the terminal voltage is expressed as a sum of terms of the form

$$V_{batt} = E_0 - K\frac{Q}{Q - it}\,it - K\frac{Q}{Q - it}\,i^* + A\exp(-B\,it) - R\,i,$$

where E_0 is a constant voltage, Q is capacity, it is extracted capacity, i is current, i^* is a filtered current term, and A and B shape the exponential zone [20]. The exact algebra is less important than the behavior it captures: an initial voltage drop and a slower polarization-driven sag using a parameter set that can be fitted from a discharge curve.

They describe a practical identification procedure using manufacturer discharge data

and validation by comparing simulated voltage profiles to those curves [20]. The limitations are clearly stated: the model does not include hysteresis and does not explicitly model temperature dependence, so it is best interpreted as a mission-level model rather than a cell-physics model [20].

For electric UAS preliminary design, the value of this model class is that it exposes voltage sag and usable-energy sensitivity under power draw without requiring stiff electrochemical dynamics. That allows design trade studies to capture the coupling between propulsion sizing, mission profile, and energy margin.

2.9 Structural Sizing in Preliminary Design

The structural sources referenced in this review are used as engineering standards rather than as novel research contributions, but they define the baseline for what structural checks look like in early aircraft design.

Timoshenko and Gere provide core stability and elastic buckling foundations [24]. Bruhn and Niu provide aircraft-oriented sizing guidance and worked examples that connect load cases to practical thickness and stiffener decisions [30, 33]. NASA SP-8007 is a standard reference for thin-walled buckling behavior and is commonly used to benchmark buckling checks [29]. In a preliminary design tool, these references support a fast check set: stress, deflection, torsion, and primary panel and column buckling limits, coupled to aerodynamic spanwise loading.

2.10 Synthesis: Requirements for a Useful Flying-Wing Design Tool

Across the sources, the theme is not that one method wins. It is that a useful preliminary design tool must make iteration cheap while staying honest about assumptions.

Nguyen et al. show a specific pattern for inserting higher-fidelity corrections into an MDO loop without destroying runtime: use DOE, fit a metamodel, and keep the optimizer in the corrected low-order space [1]. Yu et al. and Rosas-Cordova et al. show that solver

comparisons only mean something when numerical convergence is treated as part of the setup, and that mesh parameters can dominate error if they are implicit [14, 15]. Zhang et al. quantify that stability constraints are not free and can impose a real performance penalty [10]. Sharpe provides a blueprint for optimization-first analysis, but also makes the engineering cost visible: analysis functions must be robust enough for an optimizer to push on them [4]. Tremblay et al. provide a battery model that is simple enough for mission simulation yet detailed enough to expose voltage sag and usable energy sensitivity [20].

If the program goal is a design environment that produces actionable insights, the remaining gaps are mostly about trust and traceability: explicit conventions (signs, axes, reference areas), repeatable numerics (mesh and panel convergence policies), and automated cross-checks against at least one independent method. Without that, the tool produces fast answers that are hard to defend when the configuration changes or when the design is handed off to detailed analysis.

Chapter 3

Implemented Software Architecture and Physical Methods

The literature review establishes what a useful flying-wing design environment must accomplish. This chapter turns from that external context to the implemented repository itself. The emphasis is on the code-backed architecture, the principal data contracts, and the physical methods already embodied in the software. This level of detail is warranted because the thesis contribution lies in the integration of disciplines across one computational workflow, not in a single standalone model.

Two design choices define the present implementation. First, the major tasks are routed through a shared project representation that can be serialized, reloaded, and extended. Second, analysis and export are treated as services acting on that representation rather than as independent scripts with ad hoc file conventions. The result is a repository that behaves less like a loose collection of utilities and more like a preliminary-design environment with explicit internal contracts.

3.1 System Architecture Overview

3.1.1 Main Layers

The repository is organized into a small number of clear layers that separate user interaction, persistent state, computational services, and downstream artifact generation. This separation is intentional. It allows the software to grow without collapsing into a collection of tightly coupled scripts.

- **GUI layer**: `app/` tabs and main window

Figure placeholder
`system_architecture_overview.png`

Figure 3.1: System architecture of the Flying Wing Tool.

- **Domain/state layer**: `core/` models and serialization
- **Service layer**: `services/` (analysis, propulsion, mission, structure, export)
- **Export/generator layer**: `core/*_gen.py` and `services/export/`

3.1.2 Primary Data Object

The central data contract is `core.state.Project`. Its role in the architecture is straightforward but fundamental: it is the mechanism by which geometry, analysis, mission assumptions, and derived results remain synchronized throughout the workflow. In other words, it is the state container that makes the integrated workflow possible. The object contains:

- `wing` (planform, twist/trim parameters, airfoil interpolation, optimized twist),
- `mission` (segments/config + GUI settings),
- `analysis` (polars, metrics, structural and mission outputs).

3.2 GUI Functionality

3.2.1 Main Window

`app/main_window.py` provides the tabbed application shell, the project lifecycle actions (new/open/save), and the top-level entry point for twist optimization. In practice, it is the

user's gateway into the shared project state and the integrated design workflow.

Tabs are:

- Geometry
- Analysis
- Mission
- Structure
- Export

3.2.2 Geometry Tab Group

`app/tabs/geometry.py` is where the aircraft is primarily defined and inspected. It includes sub-tabs for:

- planform and BWB/body-section editing,
- control-surface editing,
- twist/trim input and optimization trigger,
- airflow visualization trigger,
- airfoil interpolation/inspection,
- performance metrics and polar plotting.

3.2.3 Analysis Tab

`app/tabs/analysis.py` provides airfoil sweep and contour-plotting workflows, including parameterized airfoil studies through `services/aero_analysis.py`. This extends the tool beyond whole-aircraft analysis and gives the user a way to examine section-level aerodynamic behavior within the same environment.

3.2.4 Mission Tab

`app/tabs/mission.py` exposes the mission-analysis workflow and supports:

- sweep/batch mission operations,
- mission profile setup and simulation execution,
- phase statistics tables,
- mission-oriented configuration workflows tied to propulsion and performance evaluation.

3.2.5 Structure Tab

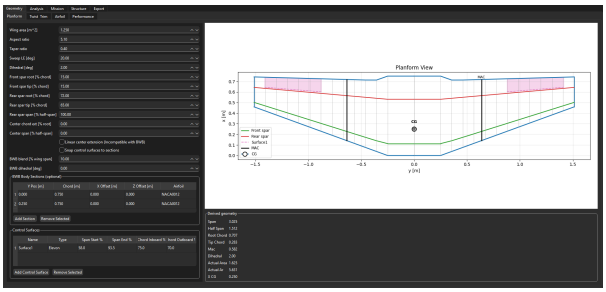
`app/tabs/structure.py` contains the structural-analysis workflow and includes:

- material and structural-parameter controls,
- structural analysis execution,
- buckling/twist/deflection visualization,
- structural optimization entry,
- PDF report export for structural results.

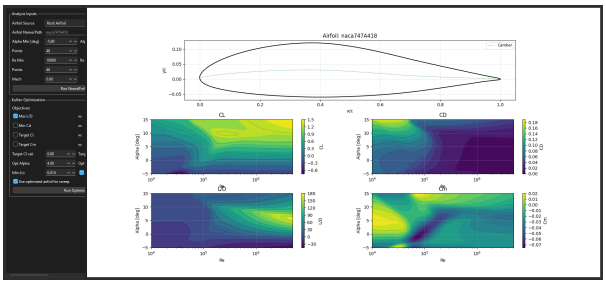
3.2.6 Export Tab

`app/tabs/export.py` provides the outward-facing interoperability layer for the GUI and supports:

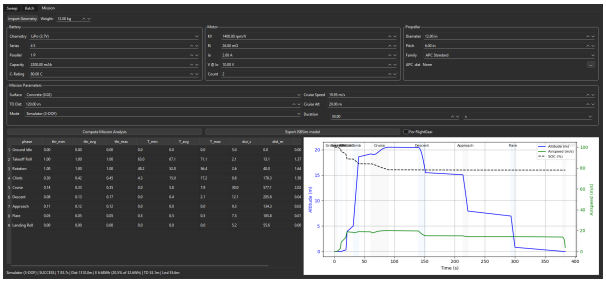
- Flow5 export,
- STEP export (including CFD wing and fixtures),
- DXF exports for ribs/spars/nested layouts with jointing to fit larger pieces on CNC,
- manufacturing split/nesting controls.



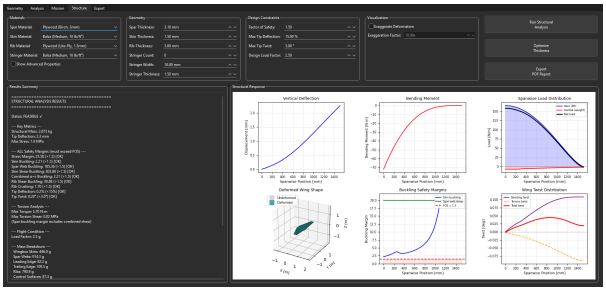
Geometry tab



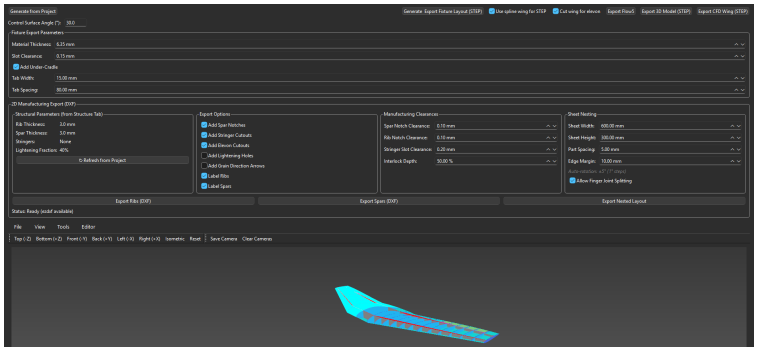
Analysis tab



Mission tab



Structure tab



Export tab

Figure 3.2: Representative GUI views of the Flying Wing Tool.

3.3 Core Analysis and Modeling Services

3.3.1 Geometry and Aerodynamics Service

`services/geometry.py` (`AeroSandboxService`) is the central aero-geometry engine of the repository. More than any other module, it embodies the thesis goal of linking geometry definition to analysis results without forcing the user to rebuild the aircraft model between stages. Its major capabilities include:

- build spanwise sections for classic flying-wing and BWB configurations,
- include control surfaces,
- compute design velocity and alpha estimates,
- generate target spanwise lift distributions (elliptical/bell logic),
- run twist optimization for trim/lift-distribution targets,
- run AeroBuildup and lifting-line analyses,
- compute mission-relevant performance metrics and polars,
- extract spanwise lift/moment distributions for structural loading,
- run aerostructural analyses using the structure service,
- build deformed wing geometry for visualization.

3.3.2 Airfoil Analysis Service

`services/aero_analysis.py` provides section-level aerodynamic utilities that complement the whole-aircraft workflow. Its capabilities include:

- robust airfoil loading (NACA generation, named/profile files, DAT parsing),
- NeuralFoil-based sweeps,

- constrained airfoil-parameter studies built around thickness, smoothness, confidence, and weighted objective terms.

3.3.3 Aero Model Abstractions

`services/aero_model.py` defines reusable aerodynamic model interfaces that allow mission and dynamics code to consume aerodynamic predictions in a consistent form. These include:

- precomputed grid-interpolated polar models,
- AD/stability-oriented model wrappers,
- rigid-body force/moment models used by mission/dynamics solvers.

3.3.4 Mission Subsystem

`services/mission/` contains the logic required to move from a static aircraft definition to a time-dependent mission simulation. The subsystem includes:

- mission schema and phase typing (`mission_definition.py`),
- autopilot logic by phase (`autopilot.py`),
- simulator orchestration and per-phase integration (`mission_simulator.py`),
- 3-DOF derivatives and numerical integration support (`dynamics_3dof.py`),
- ground-roll physics module (`ground_roll.py`),
- legacy APC map/motor/sweep helpers for propeller-performance workflows.

3.3.5 Propulsion Subsystem

`services/propulsion/` provides a coupled propulsion chain:

- battery model with SOC-dependent OCV and resistance behavior,
- ESC power and thermal loss model,

- motor model (Drela-style analytical motor relations),
- propeller coefficient model(s) including pretrained surrogate support,
- thermal network for motor/ESC/battery,
- integrated equilibrium solver producing thrust, power, temperatures, and efficiency metrics, with multi-motor support.

Pretrained propeller models are expected in `data/propeller_models/*.pkl`. This arrangement makes it possible to mix analytical relations and learned surrogate behavior within the same propulsion workflow.

3.3.6 Structural Subsystem

`services/structure.py` is a substantial structural-analysis module that translates aerodynamic loading into preliminary structural consequences. It implements:

- wingbox section/material/stringer/rib data structures,
- beam analysis (including Timoshenko shear-deformation effects),
- skin/spar/rib buckling checks,
- stringer crippling and column-buckling checks,
- shear/torsion stress distribution in closed wingbox sections,
- post-buckling effective-width and load-redistribution logic,
- feasibility/margin aggregation,
- thickness/stringer optimization workflows.

3.4 Export and Interoperability

3.4.1 Geometry Builder

`services/export/geometry_builder.py` is the central geometry-construction module for CAD and export workflows. It supports building:

- skins,
- spars,
- ribs (including control-surface rib handling),
- stringers,
- wingbox skin,
- control-surface solids,
- fixtures and base plates.

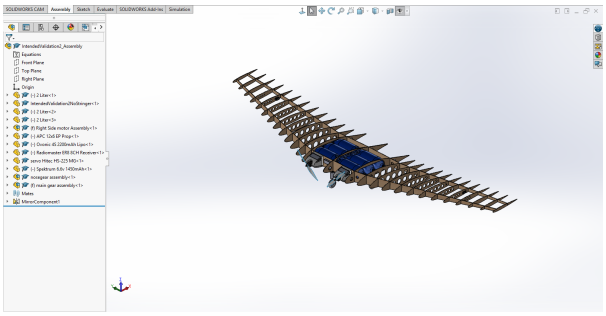
3.4.2 Target Formats

The codebase supports generation and export in multiple formats:

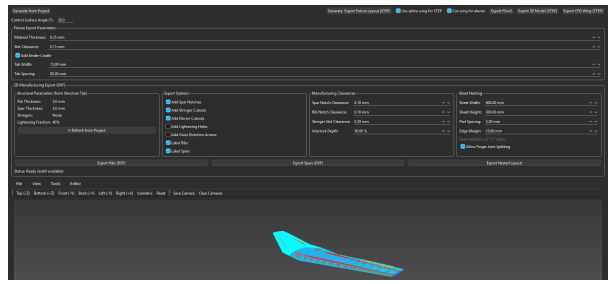
- XFLR5 / Flow5 XML plus DAT airfoils (`core/flow5_gen.py`)
- STEP and fixture geometry (`services/step_export.py`, `services/export/step_export.py`)
- DXF rib/spar/nested manufacturing layouts (`services/export/dxf_export.py`)
- JSBSim configuration and FlightGear (`core/jsbsim_gen.py` `core/flightgear_gen.py`)

3.5 Physical Methods Used

This section summarizes the principal low-order physical relations embedded in the current repository. The equations shown here are not intended as generic textbook background.



Representative CAD/isometric geometry output



Export workflow interface

Figure 3.3: Geometry and export views for interoperability workflows.

They are included because they correspond directly to the implemented models and therefore define the interpretation limits of the resulting outputs.

3.5.1 Variable Definitions (Nomenclature)

Table 3.1 defines the symbols used in the physical-method equations. Some symbols are context-dependent (for example μ in aerodynamics vs. ground friction).

Table 3.1: Primary variables and constants used in this report.

Symbol	Definition	Units	Context
ρ	Air density	kg/m^3	Aero/mission
μ	Dynamic viscosity of air (for Reynolds number)	Pa·s	Aero
g	Gravitational acceleration	m/s^2	Global
V	True airspeed magnitude	m/s	Mission
V_{air}	Effective airspeed in ground roll ($V_{ground} + V_{headwind}$)	m/s	Ground
V_{ground}	Ground speed along runway or ENU magnitude	m/s	Mission/ground
$V_{wind,e}, V_{wind,n}$	Wind components in East/North directions	m/s	Mission

Symbol	Definition	Units	Context
$V_{headwind}$	Headwind component along runway (positive = headwind)	m/s	Ground
x, y, h	East, North, and altitude position states	m	Mission
s_{ground}	Cumulative ground-track distance state	m	Mission
α	Angle of attack command/operating value	rad or deg	Aero/mission
γ	Flight-path angle	rad or deg	Mission
χ	Track angle in horizontal plane	rad or deg	Mission
ϕ	Bank angle	rad or deg	Mission
β_{brake}	Brake command ratio in $[0, 1]$	–	Ground
m	Vehicle mass	kg	Mission/structure
W	Vehicle weight ($W = mg$)	N	Mission/ground
q	Dynamic pressure ($\frac{1}{2}\rho V^2$)	Pa	Aero/mission
S	Wing reference area	m^2	Aero/mission
L	Lift force	N	Aero/mission
D	Aerodynamic drag force	N	Aero/mission
T	Net propulsive thrust force	N	Mission/propulsion
N	Normal reaction force at gear ($N = \max(0, W - L)$)	N	Ground
$F_{friction}$	Ground friction force	N	Ground
μ_{roll}, μ_{brake}	Rolling and braking friction coefficients	–	Ground
n	Used in two contexts: load factor (L/W) and prop rev/s ($\omega/2\pi$)	– or 1/s	Mission/prop
ω	Motor/prop angular speed	rad/s	Propulsion
J	Propeller advance ratio ($V/(nD_{prop})$)	–	Propulsion
C_L, C_D	Lift and drag coefficients	–	Aero
C_T, C_P	Propeller thrust and power coefficients	–	Propulsion
Q_{prop}	Propeller shaft torque	N·m	Propulsion
D_{prop}	Propeller diameter	m	Propulsion

Symbol	Definition	Units	Context
K_e, K_t	Motor back-EMF and torque constants	V/(rad/s), N·m/A	Propulsion
R	Electrical resistance (motor or battery equivalent)	Ω	Propulsion
I	Electrical current	A	Propulsion
I_0	Motor no-load current	A	Propulsion
SOC	Battery state of charge (0 to 1)	–	Propulsion/mission
OCV	Battery open-circuit voltage	V	Propulsion
V_{term}	Battery terminal voltage under load	V	Propulsion
C_{Ah}	Battery capacity in amp-hours	Ah	Propulsion
$P_{prop}, P_{shaft}, P_{battery}$	Propeller power, shaft power, battery power	W	Propulsion
y	Spanwise coordinate in structural beam model	m	Structure
$q(y)$	Distributed structural load per unit span	N/m	Structure
V (structure)	Internal shear force in beam equations	N	Structure
M	Internal bending moment	N·m	Structure
ψ	Bending rotation in Timoshenko theory	rad	Structure
w	Transverse deflection	m	Structure
γ (structure)	Shear strain in Timoshenko theory	–	Structure
EI	Bending stiffness distribution	N·m^2	Structure
GJ	Torsional stiffness distribution	N·m^2	Structure
κ_s	Shear correction factor	–	Structure
$A_{enclosed}$	Enclosed wingbox area for Bredt torsion	m^2	Structure
t, t_{skin}, t_{spar}	Generic, skin, and spar-web thicknesses	m	Structure
σ, τ	Normal and shear stress	Pa	Structure
σ_{cr}, τ_{cr}	Critical buckling stresses (normal, shear)	Pa	Structure
b_{eff}	Effective post-buckled skin width	m	Structure
K	Effective column-length factor (Euler buckling)	–	Structure

3.5.2 Aerodynamics

The aerodynamic implementation combines geometry-driven AeroSandbox analysis with lower-order fallback relations used in mission and robustness paths. At the most basic level, the force mapping follows the standard dynamic-pressure form:

$$q = \tfrac{1}{2}\rho V^2, \tag{3.1}$$

$$L = qSC_L, \tag{3.2}$$

$$D = qSC_D. \tag{3.3}$$

When a full aerodynamic model is unavailable, the fallback polar used in mission dynamics is:

$$C_L = C_{L0} + C_{L\alpha}\alpha, \tag{3.4}$$

$$C_D = C_{D0} + kC_L^2. \tag{3.5}$$

For spanwise target loading in `services/geometry.py`, the shape function is:

$$f(\eta) = \begin{cases} \sqrt{1-\eta^2} & \text{(elliptical)} \\ (1-\eta^2)^{1.5} & \text{(bell / Prandtl-D)} \end{cases} \tag{3.6}$$

with scaling chosen to match the target global C_L:

$$C_{L,\text{section}}(\eta) = \text{scale} \cdot f(\eta), \quad \text{scale} = \frac{C_{L,\text{design}}S}{2\int_0^{b/2} f(y)c(y)\,dy}. \tag{3.7}$$

The local twist relation used for target shaping is:

$$\theta_{\text{req}} \approx \frac{C_{L,\text{section}}}{C_{L\alpha,\text{section}}} + \alpha_{L=0,\text{section}}. \tag{3.8}$$

Listing 3.1: Aerodynamic load and spanwise shaping logic

```
# services/geometry.py
```

```
if params.lift_distribution == "elliptical":
    shape = [math.sqrt(max(0.0, 1 - s.span_fraction ** 2)) for s in sections]
else:
    shape = [max(0.0, 1 - s.span_fraction ** 2) ** 1.5 for s in sections]

cl_design = scale * base_shape
required_twist_absolute = cl_design / max(cl_alpha_section, 1e-6) +
    alpha_l0_section

# services/mission/dynamics_3dof.py fallback polar + forces
CL = config.CL0 + config.CLa * alpha_rad
CD = config.CD0 + config.k * CL**2
q = 0.5 * rho * V**2
L = q * S * CL
D = q * S * CD
```

3.5.3 3-DOF Mission Dynamics

The implemented mission-oriented flight model uses the states $(x, y, h, V, \gamma, \chi, \mathrm{SOC}, T_{motor}, T_{esc}, T_{battery}, s_{grou}$ and the canonical equations:

$$\dot{V} = \frac{T\cos\alpha - D}{m} - g\sin\gamma, \tag{3.9}$$

$$\dot{\gamma} = \frac{T\sin\alpha + L\cos\phi - mg\cos\gamma}{mV}, \tag{3.10}$$

$$\dot{\chi} = \frac{L\sin\phi}{mV\cos\gamma}, \tag{3.11}$$

$$\dot{x} = V\cos\gamma\sin\chi + V_{wind,e}, \tag{3.12}$$

$$\dot{y} = V\cos\gamma\cos\chi + V_{wind,n}, \tag{3.13}$$

$$\dot{h} = V\sin\gamma, \tag{3.14}$$

$$n = \frac{L}{mg}. \tag{3.15}$$

Turn response is limited using:

$$\phi = \text{clip}(\phi_{cmd}, -\phi_{max}, \phi_{max}), \tag{3.16}$$

$$\phi \leq \arccos\left(\frac{1}{n_{max}}\right) \text{ if } n_{max} \text{ is set.} \tag{3.17}$$

Wind is represented in ENU components and directly added to the air-relative velocity for kinematics.

Listing 3.2: 3-DOF derivative implementation

```
# services/mission/dynamics_3dof.py
dV_dt = (T * np.cos(alpha_thrust) - D) / m - g * np.sin(gamma_rad)
dgamma_dt_rad = (T * np.sin(alpha_thrust) + L * np.cos(bank_rad) - W * np.cos(
    gamma_rad)) / (m * V)
dchi_dt_rad = (L * np.sin(bank_rad)) / (m * V * cos_gamma)

V_air_east = V * cos_gamma * np.sin(track_rad)
V_air_north = V * cos_gamma * np.cos(track_rad)
dx_dt = V_air_east + wind_east
dy_dt = V_air_north + wind_north
dh_dt = V * np.sin(gamma_rad)
```

3.5.4 Ground Roll and Transition Physics

In `services/mission/ground_roll.py`, the along-runway one-degree-of-freedom model uses:

$$V_{air} = V_{ground} + V_{headwind}, \tag{3.18}$$

$$L = \tfrac{1}{2}\rho V_{air}^2 S C_{L,ground}, \tag{3.19}$$

$$N = \max(0, W - L), \tag{3.20}$$

$$\mu = \mu_{roll}(1 - \beta_{brake}) + \mu_{brake}\beta_{brake}, \tag{3.21}$$

$$F_{friction} = \mu N. \tag{3.22}$$

Takeoff and landing longitudinal acceleration:

$$\dot{V}_{ground} = \begin{cases} \dfrac{T - D - F_{friction}}{m} & \text{takeoff} \\ -\dfrac{D + F_{friction}}{m} & \text{landing} \end{cases} \tag{3.23}$$

The liftoff logic is:

$$V_{air} \geq V_{rot} \ \wedge \ (N \leq N_{thresh} \ \vee \ L \geq 0.95W)\,. \tag{3.24}$$

Listing 3.3: Ground roll force balance and liftoff checks

```
# services/mission/ground_roll.py
V_air = max(0.1, V_ground + headwind)
q = 0.5 * rho * V_air**2
L = q * S * CL
N = max(0.0, W - L)
mu = ground_config.mu_roll * (1.0 - brake) + ground_config.mu_brake * brake
F_friction = mu * N

if is_landing:
    accel = -(D + F_friction) / m
else:
    accel = (T - D - F_friction) / m
```

3.5.5 Propulsion Physics

The integrated propulsion chain is battery → ESC → motor → propeller, with equilibrium solved at each time step. This matters because the mission solver is not driven by a single isolated thrust curve; it is driven by a coupled electrical and mechanical model.

Propeller nondimensional relations:

$$n = \frac{\omega}{2\pi}, \quad J = \frac{V}{nD}, \tag{3.25}$$

$$T = C_T \rho n^2 D^4, \tag{3.26}$$

$$P_{prop} = C_P \rho n^3 D^5, \tag{3.27}$$

$$Q_{prop} = \frac{P_{prop}}{\omega}. \tag{3.28}$$

Motor (Drela-style BLDC model):

$$V_{motor} = K_e \omega + IR, \tag{3.29}$$

$$\tau_{motor} = K_t (I - I_0), \tag{3.30}$$

$$P_{shaft} = \tau_{motor} \omega. \tag{3.31}$$

Battery model:

$$V_{term} = OCV(SOC) - IR(SOC, T), \tag{3.32}$$

$$S\dot{O}C = -\frac{I}{3600\, C_{Ah}}, \tag{3.33}$$

$$\dot{Q}_{battery} = I^2 R. \tag{3.34}$$

The propulsion system solves $\tau_{motor}(\omega) = \tau_{prop}(\omega)$ by bounded iteration and updates loaded battery voltage consistently.

Listing 3.4: Coupled propulsion equilibrium and battery dynamics

```
# services/propulsion/propulsion_system.py
omega_low = 0.0
omega_high = omega_limit
for _ in range(max(1, n_iterations)):
    omega = (omega_low + omega_high) / 2.0
    tau_prop = self.config.propeller.get_torque(V_freestream, omega, rho)
    I_motor = (V_motor_cmd - self.motor_model.Ke * omega) / self.motor_model.R
    tau_motor = (I_motor - self.motor_model.I0) * self.motor_model.Kt
```

```
    if tau_motor > tau_prop:
        omega_low = omega
    else:
        omega_high = omega

# services/propulsion/battery_model.py
V_terminal = OCV - I_discharge * R
dSOC_dt = -I_discharge / (3600 * self.pack.capacity_Ah)
Q_heat = I_discharge**2 * R
```

3.5.6 Structural Mechanics

The structural implementation combines beam theory, closed-section torsion, buckling checks, and post-buckling logic. The resulting model is explicitly preliminary in character, but it is still rich enough to expose the dominant trends and limiting mechanisms that matter during early design iteration.

Timoshenko beam field equations:

$$\frac{dV}{dy} = -q(y), \tag{3.35}$$

$$\frac{dM}{dy} = V, \tag{3.36}$$

$$\kappa_b = \frac{M}{EI}, \tag{3.37}$$

$$\gamma = \frac{V}{\kappa_s GA}, \tag{3.38}$$

$$\frac{dw}{dy} = \psi + \gamma. \tag{3.39}$$

The shear stiffness uses:

$$\kappa_s GA \approx \kappa_s G_{spar} \left(2ht_{spar}\right). \tag{3.40}$$

Closed wingbox torsional shear (Bredt):

$$q_t = \frac{T}{2A_{enclosed}}, \quad \tau_t = \frac{q_t}{t}. \tag{3.41}$$

Orthotropic skin compressive buckling uses plate stiffnesses $D_{11}, D_{22}, D_{12}, D_{66}$:

$$N_{cr}(m) = \frac{\pi^2}{b^2}\left[D_{11}\left(\frac{mb}{a}\right)^2 + 2(D_{12} + 2D_{66}) + D_{22}\left(\frac{a}{mb}\right)^2\right], \tag{3.42}$$

$$\sigma_{cr} = \frac{\min_m N_{cr}(m)}{t}. \tag{3.43}$$

Spar web shear buckling (isotropic form used in symbolic path):

$$\tau_{cr} = k_s \frac{\pi^2 E}{12(1-\nu^2)}\left(\frac{t}{h}\right)^2. \tag{3.44}$$

Stringer and post-buckling relations:

$$\sigma_{Euler} = \frac{\pi^2 EI}{A(KL)^2}, \tag{3.45}$$

$$\sigma_{allow} = \min(\sigma_{crippling}, \sigma_{Euler}, \sigma_{yield}), \tag{3.46}$$

$$b_{eff} = b\sqrt{\frac{\sigma_{cr}}{\sigma_{applied}}} \quad (\sigma_{applied} > \sigma_{cr}). \tag{3.47}$$

Listing 3.5: Key structural equations as implemented

```
# services/structure.py (Timoshenko + torsion)
gamma = V / (kappa_GA + 1e-10)
curvature = M / (EI + 1e-10)
q_torsion = torque / (2 * A_enclosed + 1e-10)
tau_torsion_front = q_torsion / (t_front + 1e-10)


# services/structure.py (buckling + stringer)
tau_cr = k_shear * (np.pi**2 * E) / (12 * (1 - nu**2)) * (t / h)**2
sigma_euler = np.pi**2 * E * I / (A * L_eff**2)
b_eff = b * np.sqrt(sigma_cr / sigma_applied)
```

The current aerostructural workflow is one-way by default: aerodynamic loads drive the structure, and the deformed structure is not iterated back into a fully coupled loop in the default path. That simplification is appropriate for the present scope, but it should be kept in mind when interpreting results as preliminary rather than fully aeroelastic.

3.6 Data Inputs and Outputs

3.6.1 Inputs

The repository is driven by a set of inputs that define the aircraft, the operating assumptions, and the analysis context. Typical user or model inputs include:

- planform and geometry parameters,
- airfoil definitions/interpolation,
- control-surface definitions,
- structural material and thickness settings,
- propulsion system parameters,
- mission phases/environment/runway settings.

3.6.2 Outputs

From those inputs, the software produces outputs that span the main preliminary-design disciplines. These outputs include:

- aerodynamic polars and performance metrics,
- optimized twist distributions,
- structural margins and deformation fields,
- mission energy/range/endurance and phase statistics,

- exported CAD/simulation files (CPACS/Flow5/STEP/DXF/JSBSim/FlightGear artifacts).

Chapter 4

Integrated Workflow, Design Use Cases, and Contribution

4.1 Shared-State Workflow as the Core Contribution

The strongest argument for the current software is that it organizes preliminary design around continuity of state. Geometry, analysis settings, mission assumptions, and export products all derive from the same project object rather than from isolated inputs scattered across separate tools. This is particularly valuable in flying-wing design because the important reference quantities change frequently during iteration. Wing area, aspect ratio, static-margin targets, spar placement, and control-surface extent do not remain fixed as the concept matures. A unified state model reduces the chance that one subsystem is operating on stale or inconsistent geometry.

This architectural choice also improves reproducibility. A saved project file is more than a record of shape; it can retain mission configuration, analysis outputs, optimized twist distributions, structural results, and derived quantities such as center-of-gravity placement. The batch-analysis entry point reinforces this by allowing the same project file to be reloaded, analyzed, and updated without GUI-specific steps. In a thesis context, that matters because it supports traceable design studies and a credible path to future validation work.

4.2 Geometry-to-Analysis Traceability

The geometry workflow begins with the familiar parameters of conceptual aircraft design: area, aspect ratio, taper, sweep, dihedral, and section-level aerodynamic properties. It extends that baseline with centerbody and control-surface representations that are particularly

Figure placeholder
`shared_state_workflow.png`

Figure 4.1: Shared-state workflow of the Flying Wing Tool.

relevant to tailless configurations. Those inputs are then converted into spanwise sections that become the common currency of the rest of the system.

That traceability matters because the spanwise sections carry information that several disciplines need simultaneously. Aerodynamic analysis uses them to define planform and loading. Structural analysis uses them to define local chord, spar placement, and sectionwise load application. Export uses them to generate ribs, spars, lofted surfaces, and manufacturing geometry. By forcing these tasks to operate on a shared section model, the code reduces the risk that the aerodynamic wing and the manufactured wing diverge during iteration.

The same traceability makes optimization loops more meaningful. Twist optimization is not treated as a disconnected numerical exercise whose results live in a separate file or spreadsheet. The optimized twist distribution is written back into the same project state that later supports polar generation, structural loading, and geometry export. The aircraft definition therefore evolves coherently rather than being reassembled at each step.

4.3 Why Integration Matters More for Flying Wings

For conventional aircraft, some disciplinary decoupling is possible because the tail can absorb part of the trim and stability burden. Flying wings do not have that margin. A wing optimized only for induced drag may be difficult to trim. A structural layout optimized only for low mass may generate manufacturing or control-surface conflicts. A mission profile that appears feasible under simplified propulsion assumptions may fail once battery sag, thermal

limits, or mass growth are included.

The current software does not remove these tradeoffs, nor should it claim to. Its value is that it makes them visible earlier. The geometry service exposes target loading and twist synthesis. The structural service translates aerodynamic loading into mass, stress, and margin-oriented outputs. The mission service ties propulsion assumptions to energy use and performance summaries. The export service translates the same geometry into fabrication-oriented representations. This is the linked workflow the literature suggests is necessary for useful preliminary design.

4.4 Representative Design Loops Enabled by the Current Codebase

The implemented environment already supports several concrete design loops that are valuable in graduate research and early-stage development:

- A planform and twist loop, in which area, aspect ratio, taper, sweep, airfoil properties, and target loading are iterated to generate twist distributions and performance trends.
- An aerostructural loop, in which aerodynamic loading is used to estimate structural response and identify mass, stress, deflection, and feasibility implications of a geometric choice.
- A propulsion-mission loop, in which battery, motor, propeller, and mission-phase assumptions are varied to assess energy use, distance, endurance, and power demand.
- A geometry-to-manufacturing loop, in which the same aircraft definition is exported as ribs, spars, nested layouts, or CAD-style geometry for downstream fabrication or inspection.

These loops are preliminary by design. They do not eliminate the need for higher-fidelity analysis or physical test. Their value lies in reducing the cost of early iteration and in preserving consistency between engineering views of the same aircraft.

4.5 Research Contribution of the Software Artifact

The present artifact contributes to the field in a software-systems sense rather than through a single new physical theory. Its principal contribution is the construction of an integrated workflow tailored to flying-wing preliminary design. AeroSandbox provides much of the aerodynamic and optimization-oriented foundation, but the repository extends that foundation in several project-specific ways:

- a project schema that unifies geometry, mission, and analysis data;
- custom spanwise geometry construction for tailless and centerbody-inclusive aircraft layouts;
- a structural module tuned toward wingbox-based preliminary sizing with orthotropic material support;
- a propulsion and mission stack suitable for early performance reasoning;
- export logic that maps the internal aircraft representation into CAD and manufacturing formats.

In short, the thesis contribution is the integration itself: a design environment that keeps geometry, analysis, mission reasoning, and export within one consistent computational space.

Chapter 5

Representative Case Study: IntendedValidation2

5.1 Purpose of the Case Study

To move the thesis beyond an architectural description, this chapter summarizes a representative project case drawn from `IntendedValidation2.json`. The purpose of the case study is not to claim final validation of the software. Rather, it is to show how the integrated workflow behaves when a concrete aircraft definition is passed through the current analysis stack. In this thesis pass, the aerodynamic and structural outputs were regenerated with the repository's batch-analysis workflow, while the mission summary was taken from the mission results already stored in the same project file.

This case is useful precisely because it is neither trivial nor uniformly successful. It produces plausible aerodynamic performance, a coherent twist distribution, and a lightweight structural solution, yet the structural feasibility flag remains false because of a local rib-crushing margin below unity. That is the right kind of result for a preliminary-design environment. The software should not merely produce favorable headline numbers; it should expose mixed outcomes and governing constraints.

5.2 Case Definition

The `IntendedValidation2` project represents a small flying-wing configuration with a blended centerbody, a single defined elevon surface, and a bell-shaped target lift distribution. Table 5.1 summarizes the principal geometric and trim inputs relevant to the case-study discussion.

The geometry is notable for two reasons. First, it uses body sections to extend the

Table 5.1: Selected geometry and trim parameters for the `IntendedValidation2` case.

Parameter	**Value**
Reference wing area	1.25 m^2
Reference aspect ratio	5.10
Actual span including centerbody	3.025 m
Actual planform area	1.625 m^2
Actual aspect ratio	5.63
Root chord	0.707 m
Tip chord	0.283 m
Mean aerodynamic chord	0.525 m
Leading-edge sweep	20.0 deg
Dihedral	2.0 deg
Gross takeoff weight	12.0 kg
Cruise altitude	100 m
Target lift distribution	Bell
Static margin	8.0%

centerbody beyond a simple trapezoidal wing-only planform. Second, it includes a nonuniform rear spar position and a defined elevon surface, making it a suitable demonstration of the shared geometry-to-structure-to-export pipeline.

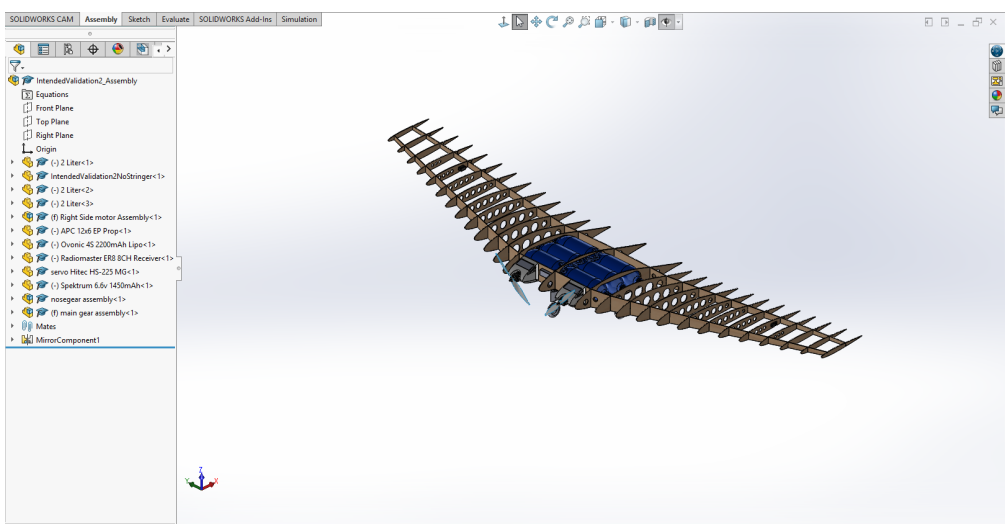


Figure 5.1: Isometric geometry view of the case-study configuration.

5.3 Aerodynamic Performance Results

The batch analysis workflow computed cruise and takeoff metrics together with full polars over an angle-of-attack range from -5 deg to 15 deg. Table 5.2 summarizes the principal operating-point results.

Table 5.2: Computed aerodynamic operating-point results for `IntendedValidation2`.

Quantity	**Cruise**	**Takeoff**
Velocity	19.95 m/s	13.34 m/s
Angle of attack	0.00 deg	5.09 deg
C_L	0.300	0.730
C_D	0.0129	0.0456
L/D	23.21	16.02
C_m	≈ 0	-0.0350

At the cruise design point, the case reaches a lift-to-drag ratio of approximately 23.2 while maintaining the target design lift coefficient near 0.30. For a compact flying-wing configuration analyzed with a preliminary method stack, that is an encouraging result. The takeoff condition is predictably less efficient, with higher drag and a lower lift-to-drag ratio, but it also delivers the higher lift coefficient needed for low-speed operation.

The batch polar sweep is valuable because it shows more than isolated operating points. It reveals how the configuration trends across angle of attack. Figure 5.2 shows the cruise and takeoff lift polars, while Figure 5.3 shows the corresponding drag trends.

The aerodynamic outputs also produce a center-of-gravity location estimate of approximately 0.250 m in the model coordinate system. Because the twist and trim settings are embedded in the same project object, that estimate remains tied to the geometry and loading assumptions that produced the operating-point results.

5.4 Optimized Twist Distribution

The same case also illustrates the twist-synthesis capability of the software. The optimized twist distribution for the project ranges from approximately +3.29 deg at the inboard end of the section set to -1.48 deg at the outboard end. Figure 5.4 shows the monotonic shift

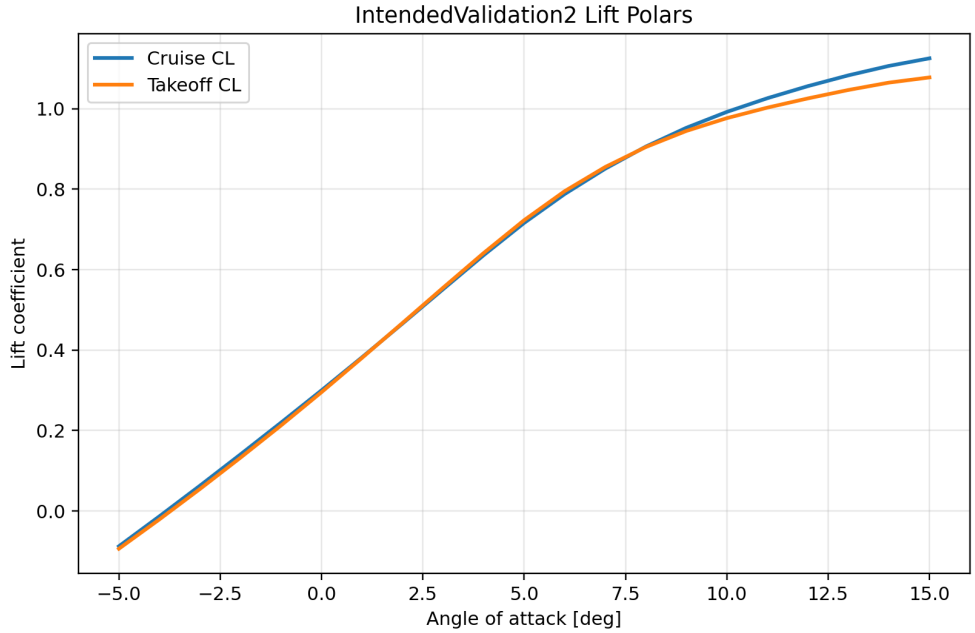


Figure 5.2: Computed cruise and takeoff lift polars for `IntendedValidation2`.

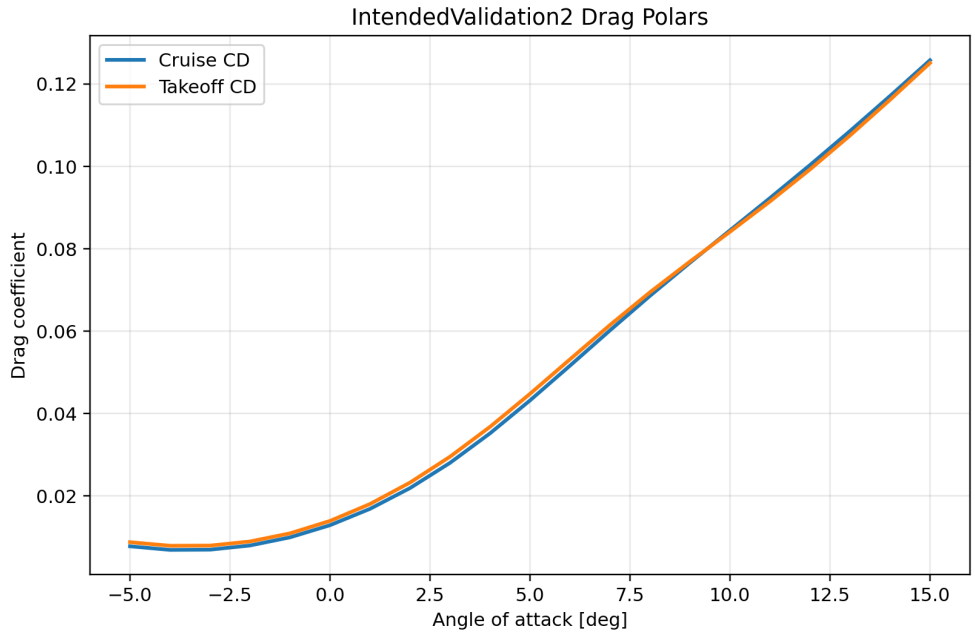


Figure 5.3: Computed cruise and takeoff drag polars for `IntendedValidation2`.

from positive inboard twist to negative outboard washout.

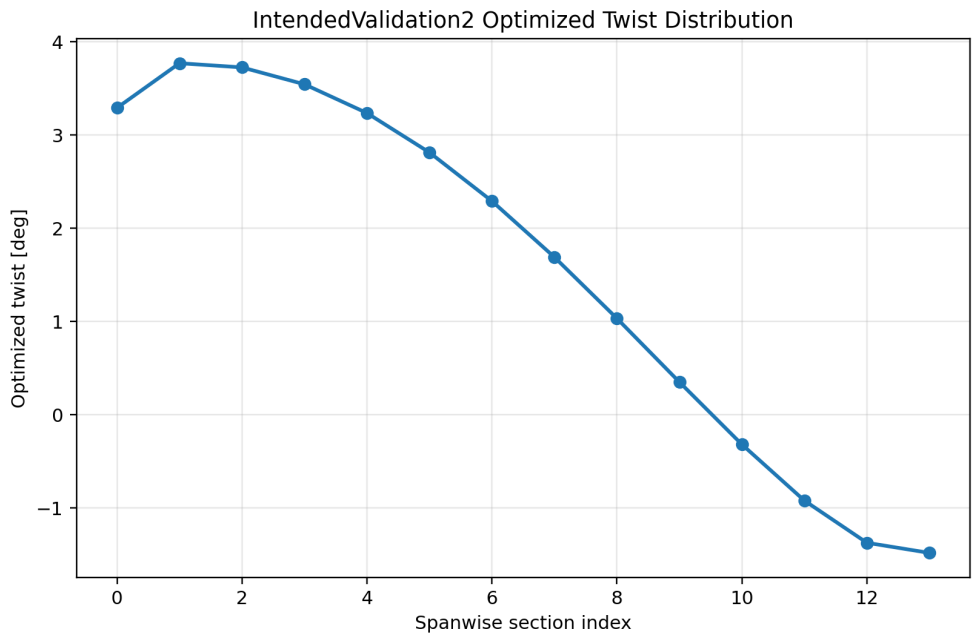


Figure 5.4: Optimized twist distribution for `IntendedValidation2`.

This result matters because it demonstrates one of the most configuration-specific parts of the workflow. Flying wings rely heavily on twist as both a trim variable and a load-shaping variable. The software therefore benefits from treating twist not as an after-the-fact geometric detail, but as a first-class design output that can immediately inform aerodynamic, structural, and export steps.

5.5 Structural Results Under a 2.5g Load Case

The structural portion of the batch analysis was run at a 2.5g load factor, consistent with the default load case in the repository's full-analysis entry point. During the aerostructural solve, the VLM path produced an unstable result and the workflow automatically fell back to an AeroBuildup-based bell-distribution load estimate before continuing. That fallback is noteworthy because it shows that the codebase already contains a practical robustness mechanism rather than assuming ideal solver behavior in every case.

Table 5.3 summarizes the principal structural outputs.

Table 5.3: Selected structural results for `IntendedValidation2` at 2.5g.

Quantity	**Value**
Estimated structural mass	2.363 kg
Tip deflection	2.27 mm
Tip twist	0.197 deg
Maximum stress	1.88 MPa
Stress margin	25.58
Minimum buckling margin	2.21
Minimum rib crushing margin	0.97
Overall structural feasibility flag	False

These results are informative because most global structural indicators are favorable. Tip deflection is very small relative to the span, tip twist is well below the maximum allowable twist, the maximum stress is low, and both the stress margin and minimum buckling margin are comfortably positive. Despite that, the overall structural feasibility flag remains false.

The reason is local rather than global. The stored structural detail shows a minimum rib-crushing margin of approximately 0.97, slightly below the threshold of unity. In other words, the structure appears broadly efficient and stiff in the beam sense, but a local bearing or crushing limit still governs. From a thesis perspective, this is exactly the kind of insight a preliminary-design environment should provide: not merely whether a design "passes," but which mechanism prevents an otherwise promising configuration from doing so.

5.6 Stored Mission Summary

The `IntendedValidation2` project file also contains a successful stored mission simulation result. That result is useful as a demonstration of the integrated mission-analysis capability, although it should be interpreted as a project-case example rather than as the centerpiece of the present rerun. Table 5.4 summarizes the top-level mission outputs retained in the project record.

The mission result is modest in range and duration, but it serves the thesis well because it shows how propulsion assumptions, mission state, and aircraft geometry can coexist inside one project definition. Even in its present preliminary form, the software can

Table 5.4: Stored mission summary associated with `IntendedValidation2`.

Quantity	**Value**
Mission success flag	True
Phases completed	9 / 9
Total mission time	82.1 s
Total distance	1283 m
Total energy used	6.64 Wh
Final state of charge	78.1%
Maximum altitude	20.5 m
Maximum speed	19.94 m/s
Maximum electrical power	2.30 kW

carry a design from planform definition through aerodynamic and structural analysis into mission-level energy accounting.

5.7 Initial BARAM CFD Cross-Check

The repository now also contains a BARAM CFD case and ParaView state for the same geometry. This is valuable because it provides an initial independent flow-solver cross-check without requiring a separate geometry rebuild. The case bundle includes a solved flow directory, monitored force outputs, solver logs, and a ParaView state with slice, contour, reflection, and stream-tracer filters already configured around the aircraft surface and wake. The corresponding solver log reports convergence after 604 SIMPLE iterations.

Several details of the stored BARAM setup matter for interpretation. The case uses an incompressible, pressure-based RANS solve with a standard k-ϵ model. The computational domain applies a symmetry plane on one side of the aircraft, so the monitored forces correspond to a half-model and must be doubled to estimate full-aircraft loads. In addition, the BARAM force-coefficient monitor was configured with $A_{\text{ref}} = 1$, $l_{\text{ref}} = 1$, and $U_\infty = 1$. The raw monitored C_L, C_D, and C_m values are therefore normalized with generic unit reference quantities rather than aircraft-specific thesis references.

To produce a meaningful engineering comparison, the monitored forces were re-scaled using the project's actual planform area of 1.625 m^2, the same air density used elsewhere in the project, and the cruise speed from the repository analysis workflow, 19.95 m/s. This

step is an inference rather than a directly stored BARAM coefficient definition, but it is a defensible one because the BARAM case is configured at zero angle of attack and the repository cruise point also occurs at zero angle of attack. The final 20 monitored iterations were averaged after doubling the half-model forces.

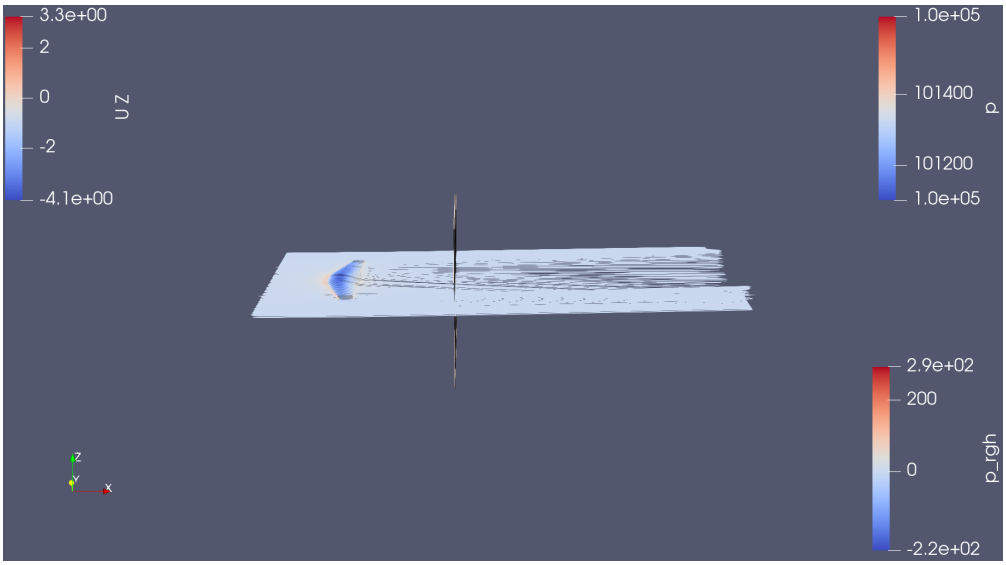


Figure 5.5: BARAM and ParaView view of the CFD cross-check case.

Table 5.5: Initial BARAM CFD cross-check against the repository cruise point for `IntendedValidation2`.

Quantity	**BARAM CFD**	**Repository cruise**	**Difference**
C_L	0.273	0.300	-0.027
C_D	0.0341	0.0129	+0.0212
L/D	8.00	23.21	-15.21
Estimated lift	108.0 N	117.7 N required	-9.7 N
Estimated drag	13.51 N	5.13 N implied	+8.38 N

Figure 5.6 shows the re-scaled coefficient convergence history, while Figure 5.7 summarizes the final comparison against the repository cruise point.

The comparison is encouraging in one respect and cautionary in another. After symmetry correction and cruise-speed normalization, the BARAM lift coefficient lies reasonably close to the repository cruise value. That suggests that the geometry, attitude, and loading assumptions are at least directionally aligned. The drag coefficient, however, is substantially larger in the BARAM result, driving the CFD-based L/D far below the low-order estimate.

Table 5.6: BARAM drag decomposition for the final 20 monitored iterations, converted to a full-aircraft estimate.

Component	Force	Share of total drag
Pressure drag	10.43 N	77.2%
Viscous drag	3.08 N	22.8%
Total drag	13.51 N	100.0%

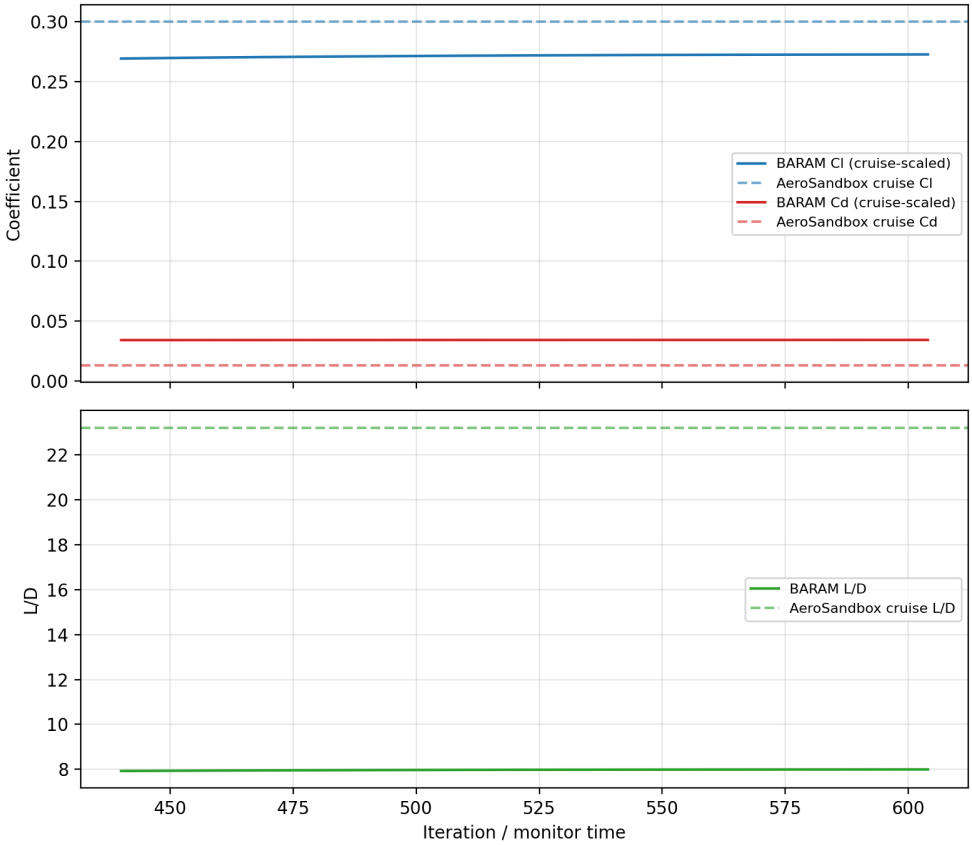


Figure 5.6: BARAM monitor history re-scaled to the `IntendedValidation2` cruise reference quantities.

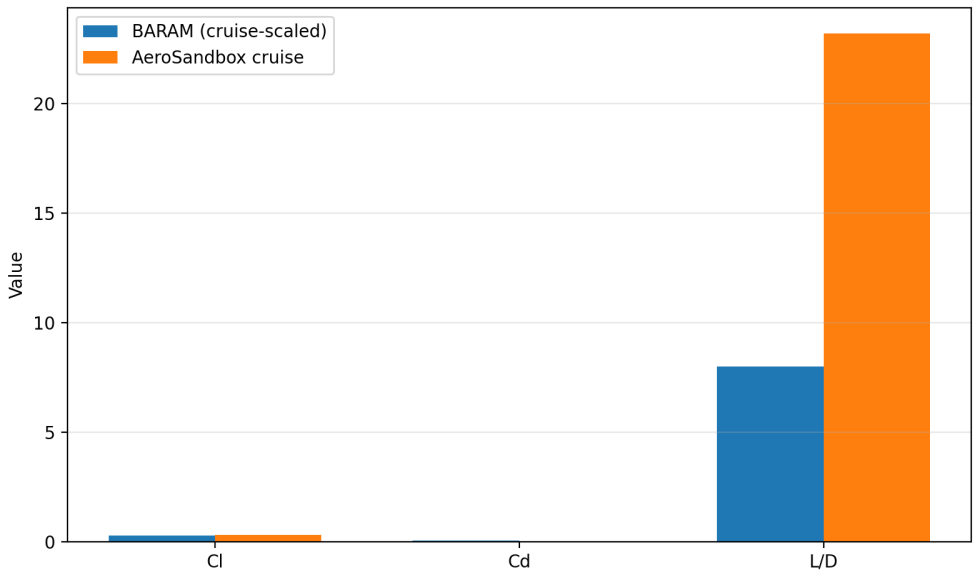


Figure 5.7: Comparison of re-scaled BARAM cruise estimates with the repository cruise point.

The force decomposition constrains the interpretation. The mismatch is not primarily explained by viscous drag alone. Table 5.6 shows that approximately 77% of the CFD drag is pressure drag and only about 23% is viscous drag. In fact, the CFD pressure-drag contribution by itself already exceeds the entire low-order cruise drag implied by the repository result. The discrepancy is therefore pressure- or form-drag dominated rather than skin-friction dominated. The most defensible conclusion is accordingly a narrow one: the present low-order workflow is under-capturing one or more pressure-drag mechanisms that the RANS case resolves. Plausible contributors include blended-centerbody form drag, interference effects, or localized separation, but identifying the dominant mechanism requires a dedicated follow-on study rather than assertion.

This result does not invalidate the repository workflow, but it does show that drag prediction and reference-condition matching require closer study before any strong accuracy claim can be made. In thesis terms, the comparison is best interpreted as an initial validation pathway and an illustration of the integrated workflow's usefulness: the discrepancy is visible, quantifiable, and traceable from a saved project directly to an external CFD case.

5.8 Interpretation of the Case Study

Taken together, the case-study results support the main argument of the thesis. The software does not simply generate geometry or isolated performance coefficients. It exposes coupled design consequences. The `IntendedValidation2` configuration appears aerodynamically attractive in the low-order workflow, with a strong cruise lift-to-drag ratio and a coherent washout distribution. The structural response is globally stiff and lightweight, but local rib crushing still prevents full structural feasibility. The stored mission result shows that the same configuration can also be examined from an energy-use standpoint without leaving the project framework. The added BARAM case extends the argument further by showing that an external CFD comparison can be tied to the same design definition and used to identify where the workflow already agrees well with higher-fidelity analysis and where it does not.

This is an instructive outcome for a thesis built around software integration. A less connected tool chain might produce a favorable aerodynamic answer and stop there. The present environment carries the design far enough to show that one local structural detail remains limiting and that an independent CFD path produces a materially different drag prediction that now requires explanation. That is precisely the sort of workflow advantage argued for in the literature review, and it is visible here in a concrete project case.

Chapter 6

Validation Strategy, Limitations, and Discussion

6.1 What the Current Repository Demonstrates

The current repository demonstrates that the software can already operate as a unified preliminary-design environment. This claim is supported by the code structure, the available entry points, the observed data flow, and compile-tested documentation. The system accepts a project definition, builds geometry, computes aerodynamic metrics, generates polars, runs structural analysis, supports mission-oriented simulation, and exports geometry products from the same underlying aircraft representation.

This is a meaningful result because the literature review shows that fragmentation is one of the main obstacles in conceptual aircraft workflows. The present codebase addresses that obstacle directly. Even before a larger validation campaign is complete, the repository establishes that an integrated software architecture for flying-wing preliminary design is both feasible and practically usable.

6.2 What Still Requires Formal Validation

What the current repository does *not* yet demonstrate is comprehensive predictive accuracy across all operating regimes or aircraft classes. Low-order aerodynamic methods remain sensitive to assumptions, especially for separated flow, post-stall behavior, and low-aspect-ratio configurations [14, 15]. Structural preliminary-sizing methods are useful for early trade studies, but they do not replace detailed finite element modeling, detailed joint analysis, or physical load testing. Mission simulation results are only as credible as the aerodynamic and

propulsion models that feed them.

The next validation layer should therefore be organized around representative case studies. Those studies should compare the tool's outputs against reference configurations, independent solvers, published data, or physical test results wherever practical. The batch-oriented project workflow already provides a useful foundation for that effort because it makes repeated, file-based studies straightforward to organize. The BARAM case introduced in Chapter 5 is valuable in this regard, but it should still be treated as an initial cross-check rather than a final validation benchmark because the monitor normalization had to be reconstructed from saved case metadata and repository cruise conditions.

6.2.1 BARAM Mesh-Sensitivity Verification

One additional verification step is especially important before the BARAM results are used to support any stronger aerodynamic claim: mesh sensitivity. The current BARAM case is already useful as an initial cross-check because it exposes a large, pressure-dominated drag discrepancy relative to the low-order cruise workflow. However, monitor convergence by itself is not sufficient to establish that the computed drag level is numerically insensitive to discretization. If the drag changes materially with mesh density, near-wall resolution, or related discretization choices, then part of the discrepancy may still be numerical rather than physical.

For that reason, a four-level mesh-sensitivity study was completed for the `IntendedValidation2` BARAM path at both the repository cruise condition and the takeoff-angle-of-attack condition. The study repeated the same geometry, solver family, domain treatment, and monitoring workflow on systematically refined meshes. Because the original "fine" mesh was only modestly larger than the medium mesh, it is relabeled here as *Medium 2*; the final refinement level, previously tracked as "extra fine," is reported as *Fine*. Table 6.1 summarizes the resulting cell counts and converged monitor-derived coefficients. A 1% coefficient-change band is retained only as an informal screening reference, not as a formal grid-independence criterion.

The cruise case is better described as reasonably stable in lift but still somewhat sensitive in drag. From Medium to Medium 2, the changes are very small, approximately

Table 6.1: Matched BARAM mesh-sensitivity results for `IntendedValidation2`, with the former fine level relabeled as Medium 2 and the former extra-fine level reported as Fine.

Condition	**Mesh**	**Cells**	C_D	C_L	L/D
Cruise	Coarse	422,082	0.0280	0.2854	10.19
Cruise	Medium	2,158,783	0.0274	0.2877	10.50
Cruise	Medium 2	2,772,883	0.0274	0.2904	10.60
Cruise	Fine	8,505,080	0.0268	0.2921	10.88
Takeoff	Coarse	422,082	0.0111	0.1660	14.99
Takeoff	Medium	2,158,783	0.0107	0.1692	15.80
Takeoff	Medium 2	2,772,883	0.0105	0.1722	16.42
Takeoff	Fine	33,900,695	0.0108	0.1712	15.83

0.02% in C_D and 0.93% in C_L. From Medium 2 to Fine, the lift change remains modest at about 0.57%, but the drag coefficient still shifts by about 2.02%. That pattern supports the conclusion that the cruise discrepancy identified in Chapter 5 is not simply a coarse-mesh artifact, while also avoiding an overstatement that the present cruise result is fully grid independent in the strict sense.

The takeoff case remains more sensitive than cruise. From Medium to Medium 2, the changes are approximately 2.02% in C_D and 1.78% in C_L. From Medium 2 to Fine, the lift change tightens to about 0.58%, but the drag coefficient still changes by about 3.13%. The takeoff-angle-of-attack solution is therefore best described as a useful mesh-sensitivity result rather than a numerically settled drag prediction.

One limitation remains important. Although the refinement ladder itself was completed, the snappyHexMesh layer-addition stage did not retain nonzero wall-layer cells on the final accepted meshes. The current study therefore establishes coefficient-sensitivity trends for the present meshing and monitoring setup, but it does not by itself establish boundary-layer independence. Accordingly, the cruise result can support the broader drag-discrepancy argument already made in this thesis, whereas stronger claims about absolute viscous-drag fidelity or wall-resolution adequacy should remain qualified until retained prism layers or an equivalent near-wall verification path are demonstrated.

6.2.2 Validated Interpretation of the Drag Discrepancy

The BARAM cross-check in Chapter 5 established two robust facts. First, the low-order cruise workflow predicts substantially less drag than the converged RANS case. Second, the discrepancy is pressure-drag dominated rather than viscous-drag dominated. Taken together, these facts already rule out a purely skin-friction explanation. The remaining question is therefore narrower: where does the excess pressure drag originate on the aircraft?

To localize that drag without changing the underlying project definition, the repository's own geometry-export stack was used to partition the half-model outer mold line into three CFD surface groups: `centerbody`, `junction`, and `outer_wing`. A split-patch continuation case was then built from the same geometry, initialized from the converged baseline flow field, and postprocessed with per-patch force monitors. For this attribution study, the patch forces are interpreted in normalized share form rather than as final thesis-level absolute coefficients. The converged unsplit BARAM case remains the authoritative source for the absolute drag coefficient; the split-patch case is used to determine where that drag is being generated.

Table 6.2: Comparison of projected planform share and split-patch CFD drag share for `IntendedValidation2`.

Region	**Planform share**	**Drag share**	**Pressure-drag share**	**Drag/area ratio**
Centerbody	23.0%	92.3%	92.3%	4.01
Junction	9.2%	1.9%	1.9%	0.21
Outer wing	67.8%	5.7%	5.8%	0.08
Centerbody + junction	32.2%	94.3%	94.2%	2.93

Table 6.2 makes the central result difficult to dismiss as a minor numerical artifact. The body-dominated region occupies only about 32% of the projected planform area, yet it carries roughly 94% of the drag magnitude in the stabilized split-patch continuation. The centerbody alone occupies about 23% of the planform but carries more than 92% of the drag magnitude. By contrast, the outer wing contributes nearly 68% of the planform area but only about 6% of the drag magnitude in the same attribution study. In other words, the discrepancy is not centered in the outer lifting surface. It is concentrated overwhelmingly in

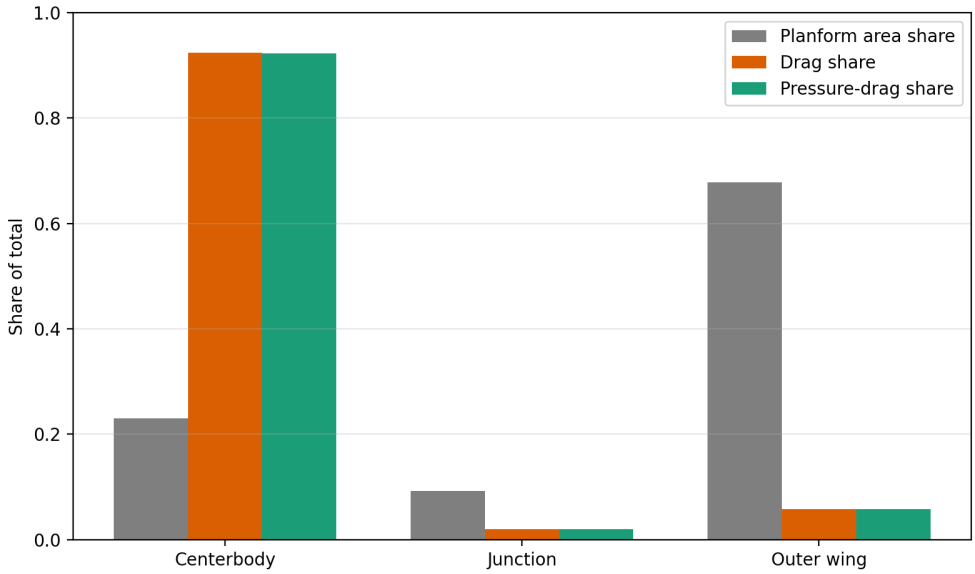


Figure 6.1: Planform-area share versus split-patch drag share for `IntendedValidation2`.

the blended central body region.

That result explains why the CFD and low-order predictions can remain relatively close in lift while diverging sharply in drag. In the present implementation, the low-order cruise analysis constructs a single symmetric AeroSandbox wing from spanwise sections and evaluates it with AeroBuildup. This is an efficient preliminary-design representation, but it does not introduce a distinct body-volume element or a dedicated wing-body interference model at the cruise-analysis stage. Similarly, when VLM fails during aerostructural analysis, the code retains total lift from AeroBuildup and then imposes an analytic bell or elliptical spanwise load shape rather than resolving a body-specific pressure field. These are reasonable approximations for rapid design iteration, but they also mean that the blended centerbody is treated primarily as part of a lifting-surface definition rather than as a volumetric source of pressure drag.

The scale of the `IntendedValidation2` case makes that omission more important, not less. At the repository cruise speed of 19.95 m/s, the root-chord Reynolds number is approximately 1.02×10^6, the tip-chord Reynolds number is approximately 3.86×10^5, and the mean aerodynamic chord Reynolds number is approximately 7.92×10^5. That is precisely the regime where a thick blended centerbody, short body-to-wing transition, and locally adverse

pressure recovery can produce large configuration-specific pressure drag even if the overall lift coefficient remains close to the low-order estimate. This is consistent with the observed pattern: the CFD lift coefficient remains within about 0.03 of the repository cruise value, but the drag coefficient is much larger.

The validation result can therefore be stated more precisely than before. For small blended flying-wing configurations of the type represented by `IntendedValidation2`, the major source of the present drag discrepancy is neither outer-wing viscous drag nor a generic failure of the entire low-order workflow. It is the under-representation of centerbody-dominated pressure drag in a wing-only preliminary aerodynamic model. This is a stronger and more useful conclusion than simply saying that "AeroBuildup is limited," because it identifies both the geometric regime in which the discrepancy appears and the part of the configuration that must be modeled more faithfully in future work.

6.2.3 Why a DATCOM-Like Buildup Method Is Weak for This Configuration

It is useful to state explicitly what kind of limitation is being observed here. The issue is not that a DATCOM-like aerodynamic buildup method is universally unreliable. Such methods are often very effective in preliminary design when the dominant behavior can be represented as the superposition of attached-flow lifting-surface trends, induced-drag relations, and moderate empirical corrections. The present case becomes difficult because the configuration combines three features that push beyond that sweet spot: a thick blended centerbody, a short transition between body and lifting surface, and a Reynolds-number range where pressure recovery is sensitive to local geometry.

The repository's cruise path makes that distinction concrete. In the current implementation, the aircraft passed to the cruise solver is built as a single symmetric `asb.Wing` assembled from spanwise sections and then wrapped in an `asb.Airplane` that contains that wing alone. This is efficient and appropriate for rapid conceptual iteration, but it means that the central body is represented primarily through changes in local section geometry, chord, and planform rather than as a separate three-dimensional body whose bluffness, shoulder

curvature, aft taper, and local crossflow can generate their own pressure-drag mechanisms. In other words, the low-order model sees a continuous lifting surface, whereas the CFD solver sees a lifting surface embedded in a volumetric flow field.

That difference matters most for drag, not lift. A section-based buildup can still recover a reasonable global lift coefficient because the integrated circulation level is driven mainly by planform, twist, angle of attack, and operating point. This is consistent with the observed results: the CFD and low-order cruise lift coefficients are relatively close. Drag, however, is much more sensitive to whether the method resolves how the flow accelerates around the centerbody shoulder, how the pressure recovers over the aft body blend, whether the body-to-wing transition creates interference losses, and whether weak local separation develops in those regions. Those are strongly three-dimensional pressure-field effects. They are precisely the kind of mechanism that a DATCOM-like buildup method does not resolve directly.

The stabilized split-patch attribution supports that interpretation. If the discrepancy were mainly due to induced-drag tuning or to generic skin-friction error on the wing, the outer wing would be expected to carry a large share of the missing drag. It does not. Instead, the centerbody dominates the drag budget despite occupying a minority of the projected planform area. That pattern is much more consistent with under-resolved body and interference pressure drag than with a simple error in outer-wing lift distribution.

The aerostructural fallback logic reinforces the same point. When VLM fails in the structural workflow, the implementation keeps total lift from AeroBuildup and then applies a prescribed bell or elliptical spanwise distribution. That is a practical and defensible fallback for structural loading studies, but it replaces local flow physics with an imposed load shape. For structure this may be acceptable at the preliminary stage; for drag attribution it is not. The pressure-drag discrepancy uncovered by the CFD comparison therefore reflects a real modeling boundary in the current low-order stack: the workflow preserves overall loading trends better than it captures centerbody-specific drag mechanisms.

For the thesis, this distinction matters because it avoids overstatement in both directions. It would be inaccurate to claim that the low-order method is simply "wrong." It would also be inaccurate to claim that the discrepancy is an unexplained CFD artifact. The more

defensible statement is narrower: for small blended flying wings with substantial centerbody volume, a DATCOM-like wing-oriented buildup remains useful for rapid lift and trim estimation, but it is not sufficient by itself for high-confidence drag prediction because the dominant missing terms are three-dimensional body and interference pressure-drag effects.

6.3 Accuracy Boundaries and Solver Assumptions

The literature reviewed earlier makes clear that early-stage aircraft software must be judged not only by whether it runs, but also by where its results are trustworthy. The current tool inherits both the strengths and the boundaries of its numerical backends. AeroSandbox-based methods support rapid iteration and optimization-oriented workflows, but their interpretation still depends on the regime and the model used [4, 8]. NeuralFoil-based airfoil sweeps provide speed and coverage, but they remain part of a preliminary-design stack rather than a replacement for higher-fidelity validation [9]. Potential-flow comparisons in the literature further reinforce the need for caution when low-order methods are extended into aggressive regimes [14].

The structural subsystem likewise operates at the preliminary-design level. Its role is to expose trends, margins, and dominant constraints early enough to influence geometry decisions. That is exactly the role it should play in a thesis of this type. The appropriate claim is therefore that the structural module improves the quality of early decisions, not that it solves the detailed-structure problem by itself.

6.4 Relevance to Graduate Research and Design Practice

The software described in this thesis is especially relevant to graduate research because it creates a practical platform for iterative study. A student or researcher can define a concept once, run multiple analyses, refine assumptions, and keep the design traceable across iterations. That is valuable in thesis work because the artifact becomes both a design tool and a research object. The same software can support literature-informed methodology,

exploratory trade studies, and later validation.

The package is also relevant to small-team design practice. Early-stage UAV development often faces the same constraints that motivated this project: limited resources, the need for rapid iteration, and a desire to avoid manually rebuilding the aircraft model every time a design decision changes. The codebase already points toward that use case by combining GUI accessibility with scriptable analysis entry points and export pathways.

6.5 Discussion of Scope Discipline

One of the recurring risks in software theses is allowing the document to drift into a proposal for everything the project *might* become. This draft deliberately resists that tendency. The manuscript claims only what is presently visible in the repository and broadly supported by the code. That discipline matters because it preserves the credibility of the eventual final thesis. A narrower, defensible claim is preferable to a broad but weakly supported one.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This thesis documents the Flying Wing Tool as an integrated preliminary-design environment for tailless aircraft. The current codebase already contains a substantial and coherent software foundation: parametric geometry definition, spanwise section generation, twist-distribution synthesis, AeroSandbox-based aerodynamic analysis, orthotropic wingbox structural assessment, mission and propulsion modeling, and direct export to CAD and manufacturing-oriented formats. The central accomplishment of the work is the integration of these capabilities around one shared project model.

That integration addresses an important practical gap in early-stage flying-wing design. Instead of separating geometry, performance reasoning, structural checks, and export into disconnected workflows, the software keeps them linked. For a configuration as tightly coupled as a flying wing, that is a meaningful contribution. The artifact does not eliminate the need for higher-fidelity analysis or experimental validation, but it improves the speed, traceability, and technical quality of early design iteration.

7.2 Future Work

Subsequent thesis work should deepen the artifact rather than merely broaden it. Several natural extensions follow directly from the present repository state:

1. Build formal validation studies against published cases, independent solver outputs, or experimental data for representative flying-wing configurations.

2. Document complete end-to-end design studies using saved project files so that the workflow can be defended as a reproducible engineering method rather than only a software architecture.

3. Strengthen optimization studies around geometry, twist, and mission objectives while preserving the current discipline of only claiming validated present-day capability.

4. Expand solver verification and data-contract checking across services so that units, assumptions, and handoff conventions remain explicit as the codebase grows.

5. Mature advanced dynamic and interoperability paths where needed, especially where external simulation environments or richer closed-loop studies are desired.

These next steps are appropriate because the repository already contains the integrated backbone required to support them. In other words, the present stage of the project has crossed an important threshold: the main question is no longer whether a unified flying-wing preliminary design environment can be built, but how far it can be validated, hardened, and extended.

References

Bibliography

[1] Nguyen, N. V., Gilard, V., and Bingol, O. (2013). Multidisciplinary Unmanned Combat Air Vehicle system design using Multi-Fidelity Model. *Aerospace Science and Technology*. doi:10.1016/j.ast.2012.12.010.

[2] Raymer, D. P. (2018). *Aircraft Design: A Conceptual Approach* (6th ed.). AIAA.

[3] Torenbeek, E. (1982). *Synthesis of Subsonic Airplane Design*. Delft University Press.

[4] Sharpe, P. D. (2021). *AeroSandbox: A Differentiable Framework for Aircraft Design Optimization*. Master's thesis, Massachusetts Institute of Technology.

[5] Bowers, A. H., et al. (2023). Wake-Structure of the Prandtl-D. AIAA SciTech.

[6] Lafifi, M. M. (2020). XFLR5 Analysis of foils and wings operating at low Reynolds numbers.

[7] Drela, M. (1989). XFOIL: An Analysis and Design System for Low Reynolds Number Airfoils. In *Low Reynolds Number Aerodynamics* (Conference Proceedings).

[8] Sharpe, P. D. (2024). *Accelerating Practical Engineering Design Optimization with Computational Graph Transformations*. Ph.D. thesis, Massachusetts Institute of Technology.

[9] Sharpe, P. D. (2025). NeuralFoil: A physics-informed machine learning model for airfoil aerodynamics. arXiv:2503.16323.

[10] Zhang, Y., et al. (2025). Aerodynamic Design Optimization for Flying Wing Gliders Based on the Combination of Artificial Neural Networks and Genetic Algorithms. *Aerospace*.

[11] Botero, E., et al. (2025). Current Capabilities and Challenges of NDARC and SUAVE for eVTOL Aircraft Design and Analysis. Stanford University.

[12] Vegh, J. M., Botero, E., and coauthors (2019). Comparison of NDARC and SUAVE for an eVTOL aircraft conceptual design: Kitty Hawk Cora. Stanford University (SUAVE).

[13] Deperrois, A. (2009). *Guidelines for XFLR5 v6.03*. XFLR5 documentation.

[14] Yu, A., et al. (2022). Comparing Potential Flow Solvers for Aerodynamic Characteristics Estimation of the T-Flex UAV. *ICAS 2022*.

[15] Rosas-Cordova, A., et al. (2024). Validation of VSPAERO for Basic Wing Simulation. *RIMNI*.

[16] Szabo, A., et al. (2022). Comparing Potential Flow Solvers for Aerodynamic Characteristics Estimation of the T-Flex UAV. ICAS.

[17] Drela, M., and Youngren, H. (1988). *AVL User Primer*. MIT AeroAstro.

[18] TechScience. (2024). Validation of VSPAERO for Basic Wing Simulation. RIMNI.

[19] Drela, M. First-Order DC Electric Motor Model.

[20] Tremblay, O., Dessaint, L.-A., and Dekkiche, A. (2007). A Generic Battery Model for the Dynamic Simulation of Hybrid Electric Vehicles. *IEEE Vehicle Power and Propulsion Conference*. doi:10.1109/VPPC.2007.4544139.

[21] Incropera, F. P., and DeWitt, D. P. *Fundamentals of Heat and Mass Transfer*.

[22] Roskam, J. *Airplane Flight Dynamics and Automatic Flight Controls*.

[23] Stevens, B. L., and Lewis, F. L. *Aircraft Control and Simulation*.

[24] Timoshenko, S. P., and Gere, J. M. *Theory of Elastic Stability*.

[25] Batdorf (1947). NACA TN-2661.

[26] Kreisselmeier, G., and Steinhauser, R. (1979). Systematic control design by optimizing a vector performance index.

[27] Lekhnitskii (1968). *Anisotropic Plates*.

[28] Whitney (1987). *Structural Analysis of Laminated Anisotropic Plates*.

[29] NASA SP-8007. Buckling of Thin-Walled Circular Cylinders.

[30] Bruhn, E. F. *Analysis and Design of Flight Vehicle Structures*.

[31] Wagner, H. (1931). Flat Sheet Metal Girders with Very Thin Webs. NACA TM 604-606.

[32] von Karman, T., Sechler, E. E., and Donnell, L. H. (1932). The Strength of Thin Plates in Compression.

[33] Niu, M. C. Y. *Airframe Structural Design*.

[34] Megson, T. H. G. *Aircraft Structures for Engineering Students*.

[35] Kulfan, B. M. (2008). Universal Parametric Geometry Representation Method. *Journal of Aircraft*, 45(1), 142–158. doi:10.2514/1.29958.

[36] Andersson, J. A. E., Gillis, J., Horn, G., Rawlings, J. B., and Diehl, M. (2019). CasADi: a software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*.

[37] Pratt, M. J. (2000). Introduction to ISO 10303 (STEP). NIST.

[38] Nagel, B., et al. (2012). Communication in Aircraft Design: Can we establish a common language? *ICAS 2012*.

[39] Berndt, J. (2009). JSBSim: An Open Source Flight Dynamics Model in C++. *AIAA Modeling and Simulation Technologies Conference*. doi:10.2514/6.2009-5699.

Chapter A

Repository-Derived Scope of the Draft

This manuscript was expanded from repository-local artifacts and code inspection rather than from speculative future plans. The main body therefore prioritizes implemented geometry, aerodynamic, structural, mission, propulsion, and export workflows. Less mature features are not presented as present-tense thesis deliverables unless they are clearly visible and defensibly bounded in the current code.

Chapter B

Primary Repository Modules Referenced in This Draft

File or Module	Role in the thesis draft
`core/state.py`	Defines the main project schema linking wing, mission, and analysis data and supporting serialization.
`core/models/planform.py`	Defines the parametric planform, control-surface, and body-section representation used throughout the workflow.
`core/models/twist_trim.py`	Stores trim and loading-target parameters used for design velocity and twist-distribution logic.
`services/geometry.py`	Implements spanwise section generation, wing construction, aerodynamic analysis entry points, performance metrics, polars, and aerostructural coupling hooks.
`services/aero_analysis.py`	Provides airfoil loading and NeuralFoil-based sweep capability for section-level studies.
`services/optimization.py`	Connects twist optimization back into the shared project state.
`services/structure.py`	Implements the wingbox beam, buckling, stringer, and mass-estimation logic used for structural preliminary sizing.
`services/mission/mission_simulator.py`	Implements mission-phase simulation, result aggregation, and output summaries.
`services/propulsion/propulsion_system.py`	Couples battery, ESC, motor, propeller, and thermal models into a propulsion analysis chain.

File or Module	Role in the thesis draft
`services/export/geometry_builder.py`	Builds geometry products used by export and CAD-style workflows.
`services/export/dxf_export.py`	Supports manufacturing-oriented export of ribs, spars, and nested layouts.
`services/step_export.py`	Supports direct STEP generation from project state.
`app/main_window.py`	Defines the top-level GUI shell and ties together the major user-facing tabs.
`app/tabs/geometry.py`	Exposes planform, control-surface, and geometry editing workflows to the user.
`app/tabs/analysis.py`	Exposes airfoil sweep and exploratory aerodynamic studies.
`app/tabs/mission.py`	Exposes mission and propulsion configuration and simulation workflows.
`app/tabs/structure.py`	Exposes structural materials, sizing controls, and structural analysis execution.
`app/tabs/export.py`	Exposes CAD, DXF, and interoperability-oriented export workflows.
`run_app.py`	Main GUI entry point.
`run_full_analysis.py`	Batch-style analysis entry point useful for reproducible file-based studies.